Exploring Multi-agent Based Human & AI Collaboration

— with the unity example of collaborative push block

Final Presentation

Members : Du,Hao Gu,Rui Lu,Haiyu Pan,Lingfeng

Date : 27/03/2025

I. Why Collaboration

II. Why "Push Block"

III. ML Agents RL

Ⅳ. Result Analysis

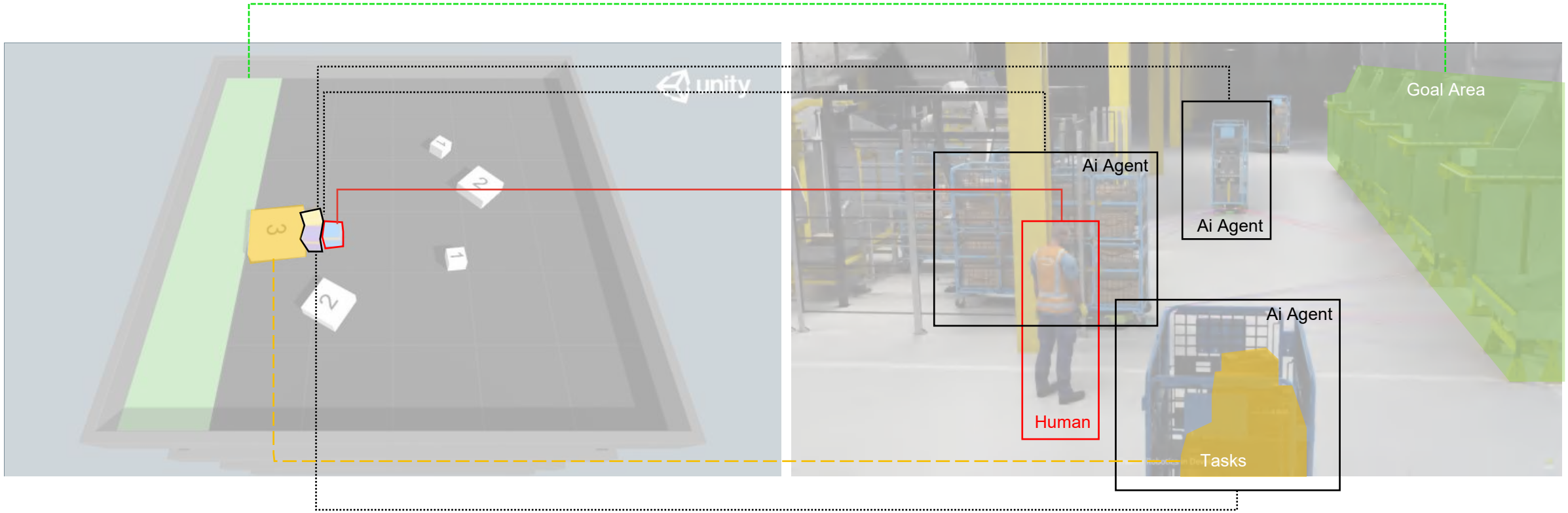
I. Why Collaboration

1. Research Background

2. Literature Review

3. Methodology

1. Research Background-WHAT



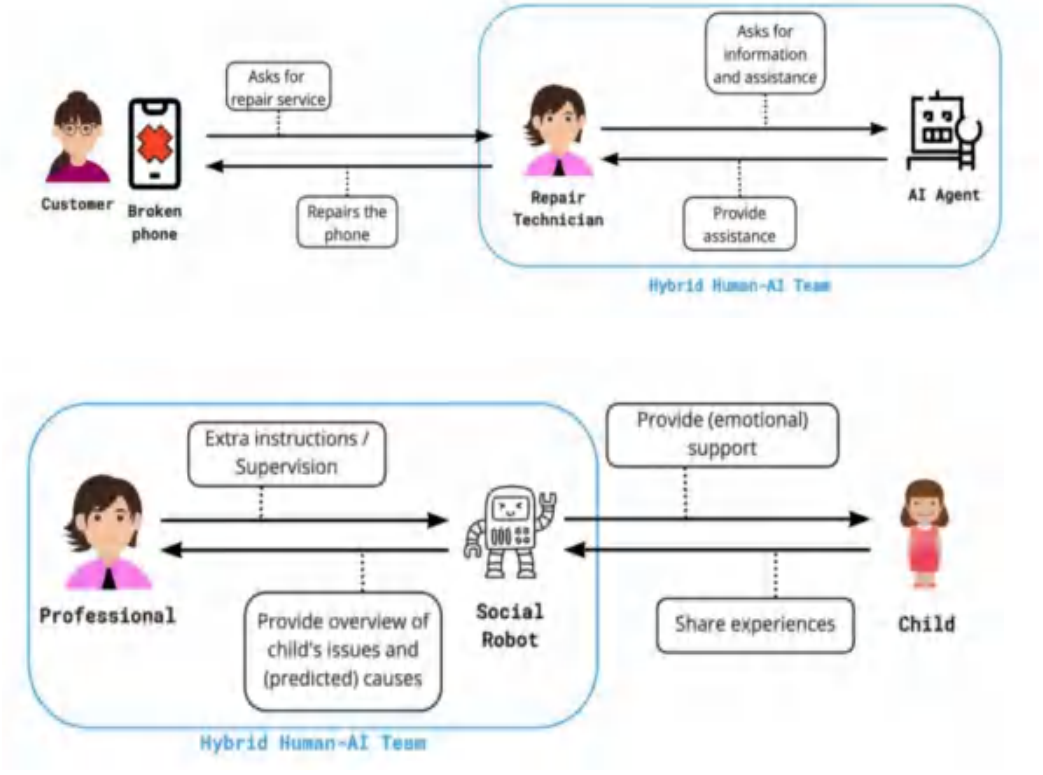
What are we doing?

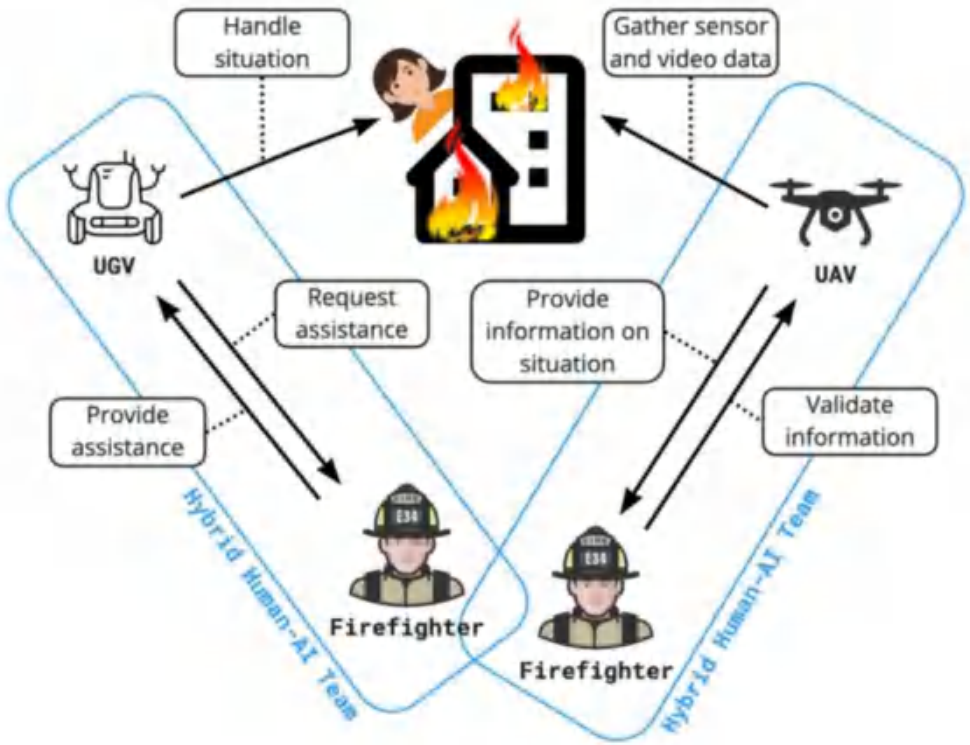
A simulation of Multi-agents/Human-Agents Collaboration Prototype

https://developer.nvidia.com/isaac-sim

https://www.amazon.jobs/content/en/teams/ftr/amazon-robotics

1. Research Background-WHY

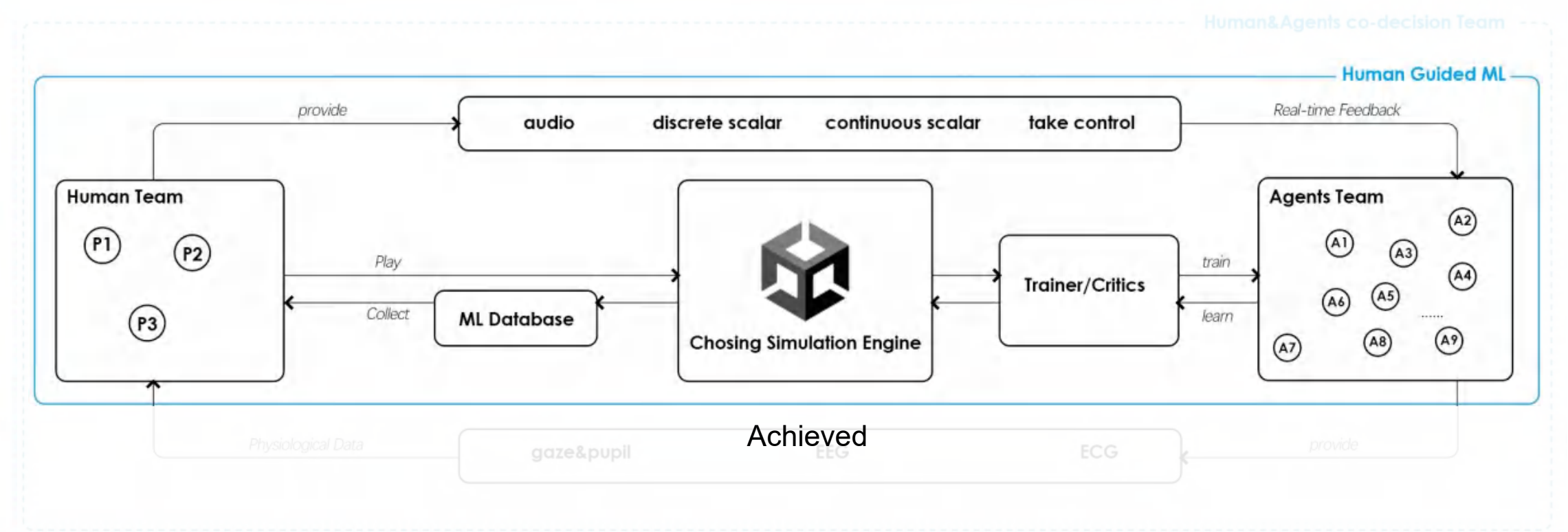




Human-AI Team Collab Senarios

Developing Team Design Patterns for Hybrid Intelligence Systems，HHAI 2023: Augmenting Human Intellect P. Lukowicz et al. (Eds.)

1. Research Background-WHY

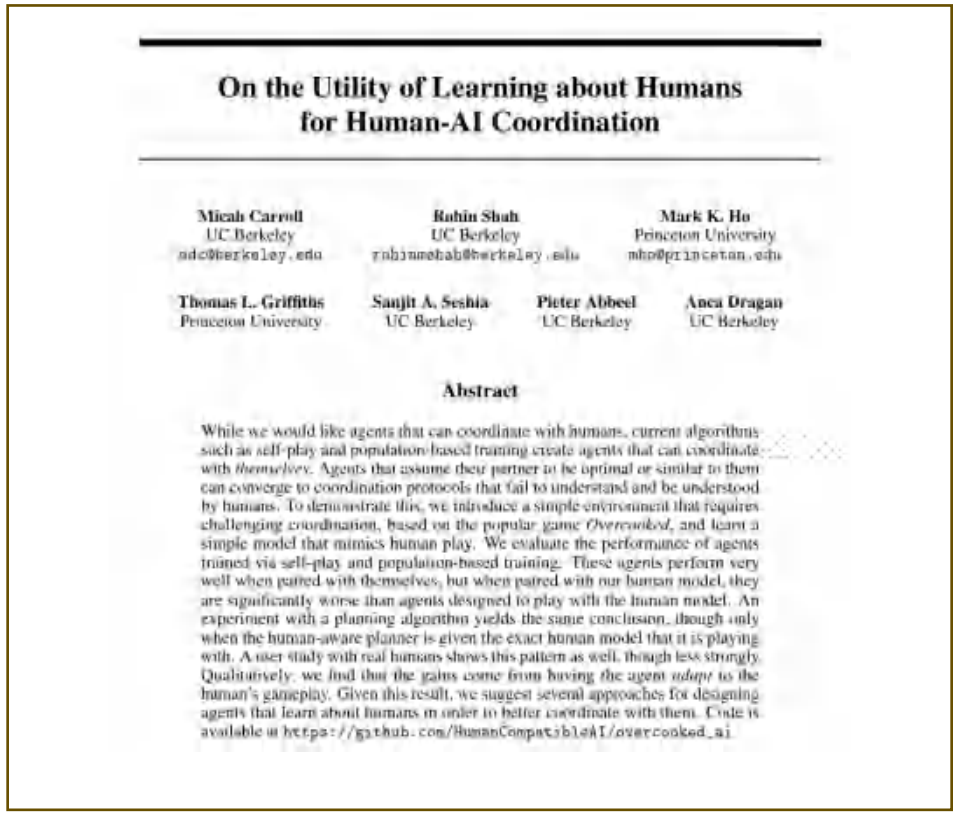


Achieved

Abstracted Human-Agent Collaboration **Simulation Prototype**

2. Literature Review-Overcooked Platform

Keywords:



On the Utility of Learning about Humans for Human-AI Coordination

Micah Carroll, UC Berkeley, mdc@berkeley.edu
Rohin Shah, UC Berkeley, rohinmshah@berkeley.edu
Mark K. Ho, Princeton University, mho@princeton.edu
Thomas L. Griffiths, Princeton University
Sanjit A. Seshia, UC Berkeley
Pieter Abbeel, UC Berkeley
Anca Dragan, UC Berkeley

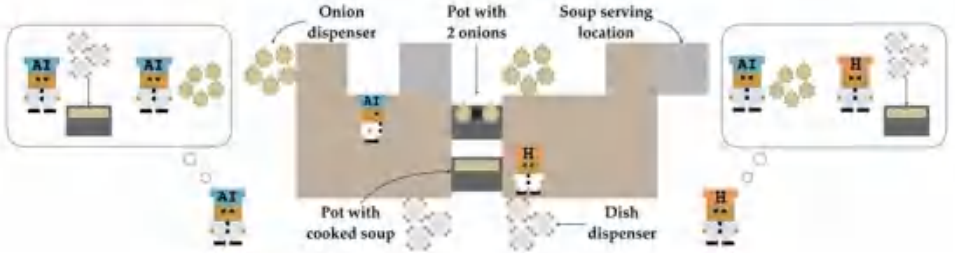
Abstract

While we would like agents that can coordinate with humans, current algorithms such as self-play and population-based training create agents that can coordinate with *themselves*. Agents that assume their partner to be optimal or similar to them can converge to coordination protocols that fail to understand and be understood by humans. To demonstrate this, we introduce a simple environment that requires challenging coordination, based on the popular game *Overcooked*, and learn a simple model that mimics human play. We evaluate the performance of agents trained via self-play and population-based training. These agents perform very well when paired with themselves, but when paired with our human model, they are significantly worse than agents designed to play with the human model. An experiment with a planning algorithm yields the same conclusion, though only when the human-aware planner is given the exact human model that it is playing with. A user study with real humans shows this pattern as well, though less strongly. Qualitatively, we find that the gains come from having the agent *adapt* to the human's gameplay. Given this result, we suggest several approaches for designing agents that learn about humans in order to better coordinate with them. Code is available at https://github.com/HumanCompatibleAI/overcooked_ai

- Human modeling
- Behavior Cloning
- Population-based Training(PBT)
- Collaborative Planning

arXiv:1910.05789v2 [cs.LG] 9 Jan 2020

Environment:



Overcooked game mechanics

5 situations in Experiment

The **Overcooked game** is a basic environment where co-operative cooking tasks need to be completed.

The different layouts of the tasks provide **low-level coordination of movements** (avoiding collisions) and **high-level strategic collaboration** (dividing up the workload) challenges.

2. Literature Review-Overcooked Platform

Training Outcomes and Conclusion:

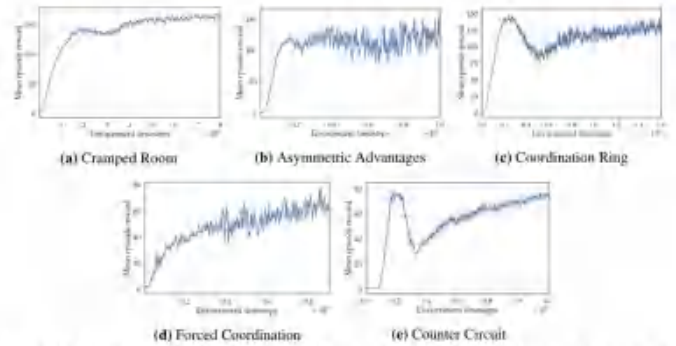


Figure 9: PPO_{BC} average episode rewards on each layout during training over 400 horizon timesteps, when pairing the agent with itself or with BC in proportion to the current self-play annealing.

Parameter	PPO_{BC} and PPO_{BC} hyperparameters				
	Cramped Rm.	Asym. Adv.	Coord. Ring	Forced Co-ord.	Counter Circ.
Learning Rate	1e-3	1e-3	1e-3	1.5e-3	1.5e-3
LR annealing factor	3	3	1.5	2	3
VF coefficient	0.5	0.5	0.5	0.1	0.1
Rew. shaping horizon	4e6	4e6	5e6	4e6	4e6
Self play annealing	[5e6, 3e6]	[1e6, 3e6]	[2e6, 6e6]	N/A	[1e6, 4e6]
# Minibatches	10	12	14	15	15
Minibatch size	1200	1000	800	800	800

PPO with embedded-agent environment

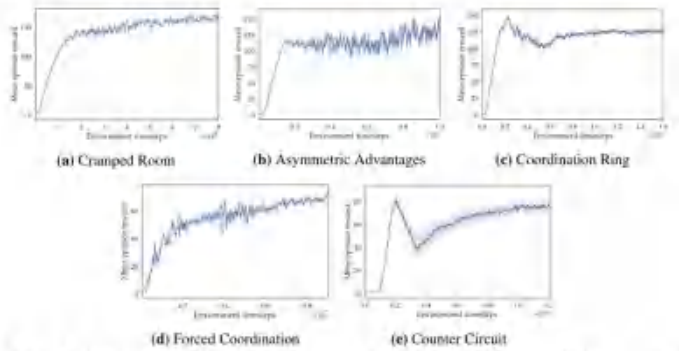


Figure 10: $PPO_{BC_{PBT+SP}}$ average episode rewards on each layout during training over 400 horizon timesteps, when pairing the agent with itself or with H_{Proxy} in proportion to the current self-play annealing.

PBT hyperparameters					
Parameter	Cramped Rm.	Asym. Adv.	Coord. Ring	Forced Co-ord.	Counter Circ.
Learning Rate	2e-3	8e-4	8e-4	3e-3	1e-3
Rew. shaping horizon	3e6	5e6	4e6	7e6	4e6
Env. steps per agent	8e6	1.1e7	5e6	8e6	8e6
# Minibatches	10	10	10	10	10
Minibatch size	2000	2000	2000	2000	2000
PPO iter. timesteps	40000	40000	40000	40000	40000

Population Based Training(PBT)

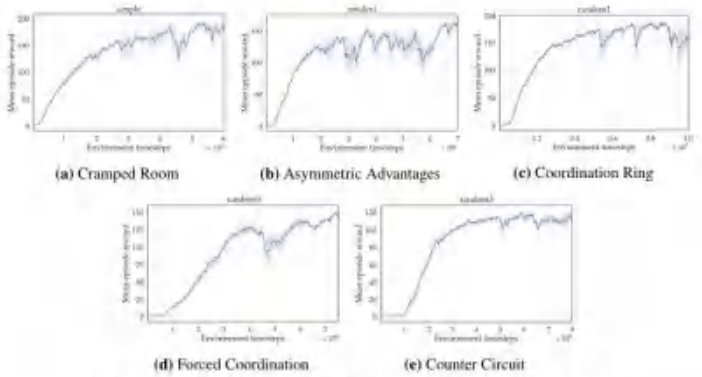


Figure 8: PPO_{SP} self-play average episode rewards on each layout during training.

PPO_{SP} hyperparameters					
Parameter	Cramped Rm.	Asym. Adv.	Coord. Ring	Forced Co-ord.	Counter Circ.
Learning Rate	1e-3	1e-3	6e-4	8e-4	8e-4
VF coefficient	0.5	0.5	0.3	0.5	0.5
Rew. shaping horizon	2.5e6	2.5e6	3.5e6	2.5e6	2.5e6
# Minibatches	6	6	6	6	6
Minibatch size	2000	2000	2000	2000	2000

Self-play PPO

- **Self-gaming and group-trained agents** showed significantly **lower performance** when working with humans, but excellent performance when working with themselves.
- Agents trained using human models with **Behavioural Clones(BC)** performed significantly better when collaborating with real humans.
- Agents that explicitly modelled human behaviour performed better in collaborative tasks.
- Human-robot collaboration scenarios cannot simply rely on self-gaming or plain old imitation learning, but should explicitly learn human behaviour models.

arXiv:1910.05789v2 [cs.LG] 9 Jan 2020

2. Literature Review-CREW Platform

Keywords:

CREW: Facilitating Human-AI Teaming Research

Lingyu Zhang, Zhengran Ji, Boyuan Chen
Duke University
{lingyu.zhang, zhengran.ji, boyuan.chen}@duke.edu

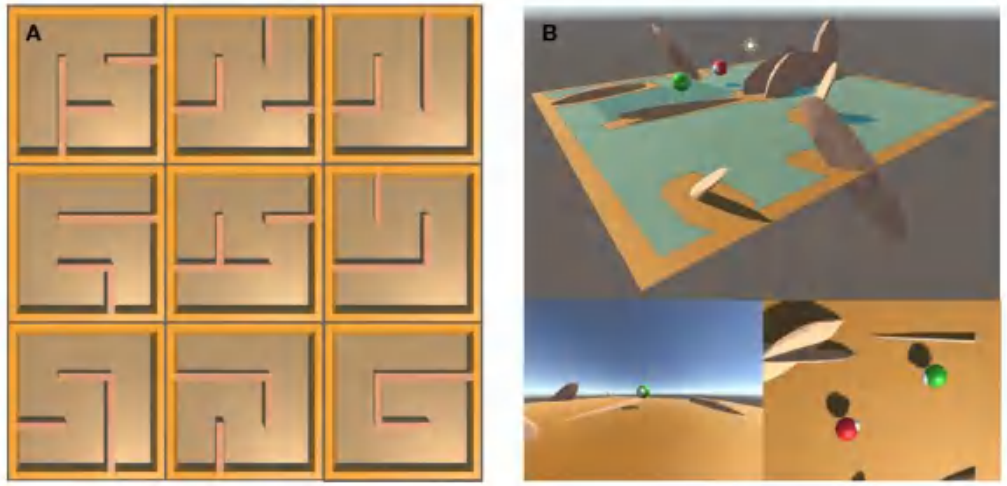
Abstract

With the increasing deployment of artificial intelligence (AI) technologies, the potential of humans working with AI agents has been growing at a great speed. Human-AI teaming is an important paradigm for studying various aspects when humans and AI agents work together. The unique aspect of Human-AI teaming research is the need to jointly study humans and AI agents, demanding multidisciplinary research efforts from machine learning to human-computer interaction, robotics, cognitive science, neuroscience, psychology, social science, and complex systems. However, existing platforms for Human-AI teaming research are limited, often supporting oversimplified scenarios and a single task, or specifically focusing on either human-teaming research or multi-agent AI algorithms. We introduce **CREW**, a platform to facilitate Human-AI teaming research and engage collaborations from multiple scientific disciplines, with a strong emphasis on human involvement. It includes pre-built tasks for cognitive studies and Human-AI teaming with expandable potentials from our modular design. Following conventional cognitive neuroscience research, CREW also supports multimodal human physiological signal recording for behavior analysis. Moreover, CREW benchmarks real-time human-guided reinforcement learning agents using state-of-the-art algorithms and well-tuned baselines. With CREW, we were able to conduct 50 human subject studies within a week to verify the effectiveness of our benchmark.

Human-AI teaming,
Real-time Human-Guided RL,
Multi-agent Task Design

https://arxiv.org/html/2408.00170v1

Environment:



Environment generation in CREW. (A) Randomized maze. (B) Procedure generated terrains.

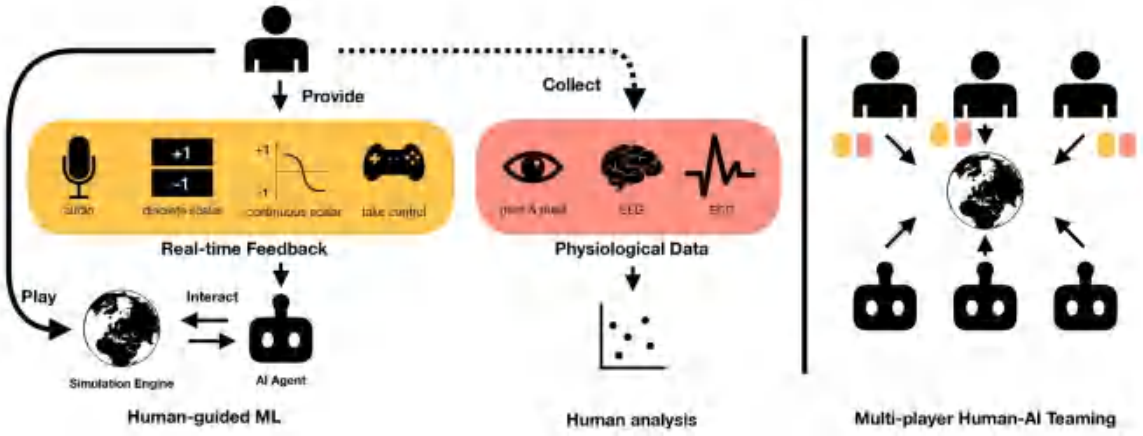
Tasks	Visual Observation	State Observation	Action Space	Reward
Bowling	grayscale	ball pos, pin pos, pin status	release pos, length before steer, steer direction	# pins hit
Find Treasure	rgb	agent pos, treasure pos	next loc x, next loc y	-1 / step, +10 treasure found
1v1 Hide-and-Seek	rgb	seeker pos, hider pos	next loc x, next loc y	-1 / step, +10 hider caught
NvN Hide-and-Seek	rgb	seekers pos, hiders pos	next loc x, next loc y	user define

Environments Settings

Developed with **Unity**.
Supports **single-player** and **multiplayer tasks**, including:
Find Treasure: Single-agent navigation task.
1v1 Hide-and-Seek: Competitive task between two agents.
NvN Hide-and-Seek: Cooperative and competitive multi-agent task with scalable complexity and agent number.

2. Literature Review-CREW Platform

Methodology:

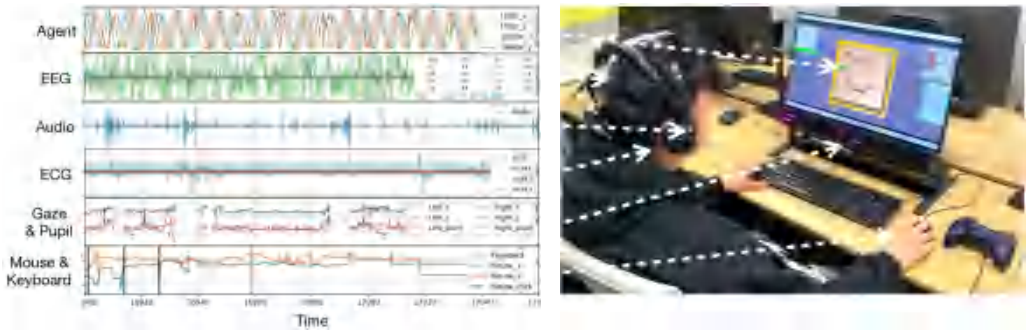


Workflow of CREW

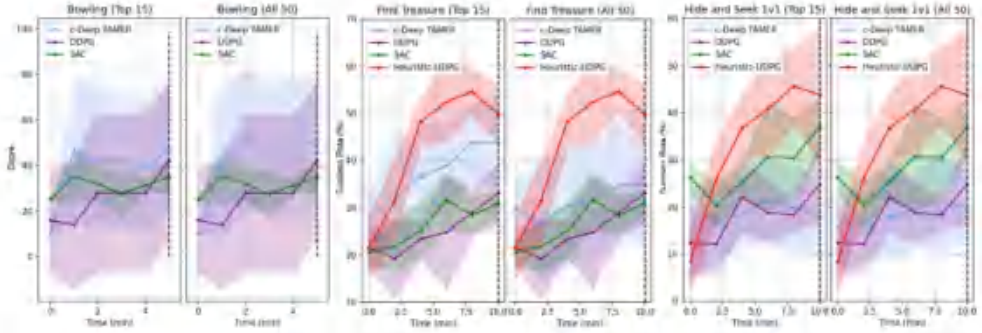
Proposes **CREW**, a unified and highly extensible platform for Human-AI collaboration research.
Supports real-time human-guided feedback (discrete and continuous).
Includes **multimodal physiological data** collection (EEG, heart rate, eye-tracking).
Benchmarked using Reinforcement Learning baselines (e.g., COACH, TAMER).

https://arxiv.org/html/2408.00170v1

Training Outcomes and Conclusion:



Physiological Data Collection



Training Curve Comparison

Provides a comprehensive platform for multidisciplinary Human-AI collaboration research.
Highlights the critical importance of **integrating real-time interaction with human cognitive and physiological data.**

2. Literature Review-MLagent Platform

Keywords:

UNITY: A GENERAL PLATFORM FOR INTELLIGENT AGENTS

Unity: A General Platform for Intelligent Agents

Arthur Juliani ARTHURJ@UNITY3D.COM
Vincent-Pierre Berges VINCENTPIERRE@UNITY3D.COM
Ervin Teng ERVIN@UNITY3D.COM
Andrew Cohen ANDREW.COHEN@UNITY3D.COM
Jonathan Harper JHARPER@UNITY3D.COM
Chris Elion CHRIS.ELION@UNITY3D.COM
Chris Goy CHRISTOPHERG@UNITY3D.COM
Yuan Gao VINCENTG@UNITY3D.COM
Hunter Henry BRANDONH@UNITY3D.COM
Marwan Mattar MARWAN@UNITY3D.COM
Danny Lange DLANGE@UNITY3D.COM
Unity Technologies
San Francisco, CA 94105 USA

Abstract

Recent advances in artificial intelligence have been driven by the presence of increasingly realistic and complex simulated environments. However, many of the existing environments provide either unrealistic visuals, inaccurate physics, low task complexity, restricted agent perspective, or a limited capacity for interaction among artificial agents. Furthermore, many platforms lack the ability to flexibly configure the simulation, making the simulated environment a black-box from the perspective of the learning system. In this work, we propose a novel taxonomy of existing simulation platforms and discuss the highest level class of *general platforms* which enable the development of learning environments that are rich in visual, physical, task, and social complexity. We argue that modern game engines are uniquely suited to act as general platforms and as a case study examine the Unity engine and open source Unity ML-Agents Toolkit[1]. We then survey the research enabled by Unity and the Unity ML-Agents Toolkit, discussing the kinds of research a flexible, interactive and easily configurable general platform can facilitate.

On the Use and Misuse of Absorbing States in Multi-agent Reinforcement Learning

Andrew Cohen*, Ervin Teng*, Vincent-Pierre Berges*, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook, Sujoy Ganguly
Unity Technologies

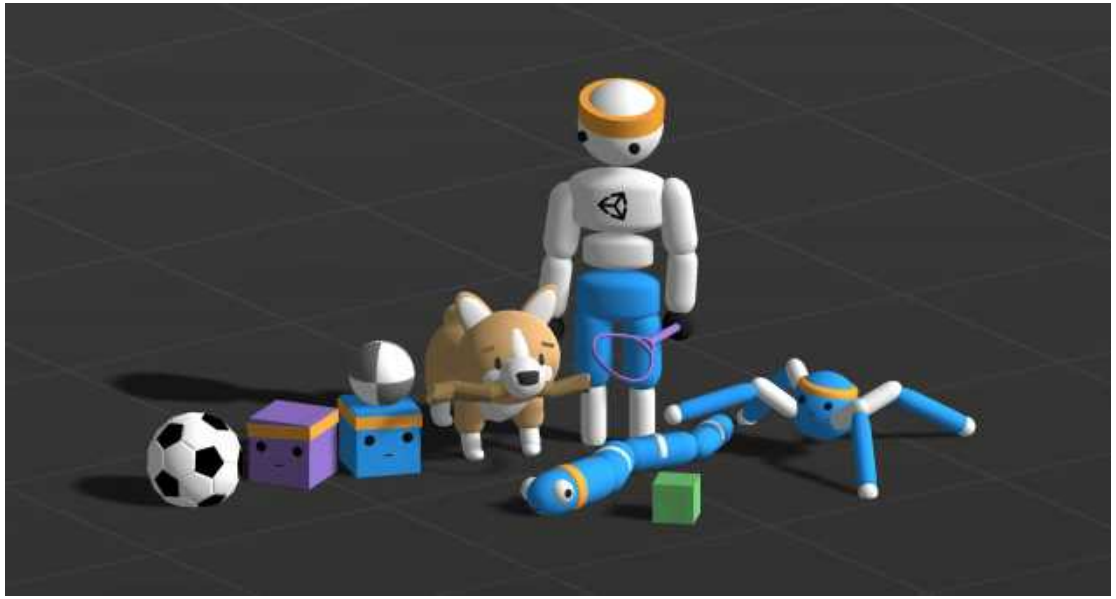
Abstract

- MARL
- Attention mechanism
- Unity ML-Agents
- Multi-agent Collaboration

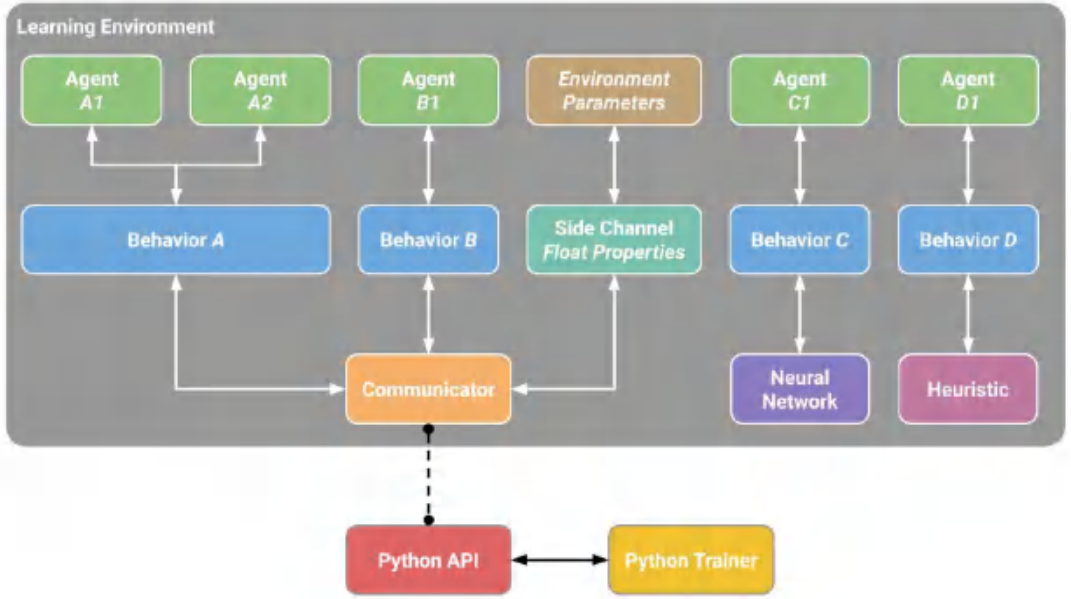
https://github.com/Unity-Technologies/ml-agents/blob/main/docs/Learning-Environment-Design-Agents.md https://unity-technologies.github.io/ml-agents/ML-Agents-Overview/

2. Literature Review-MLagent Platform

Methodology:



Mlagents Toolkit



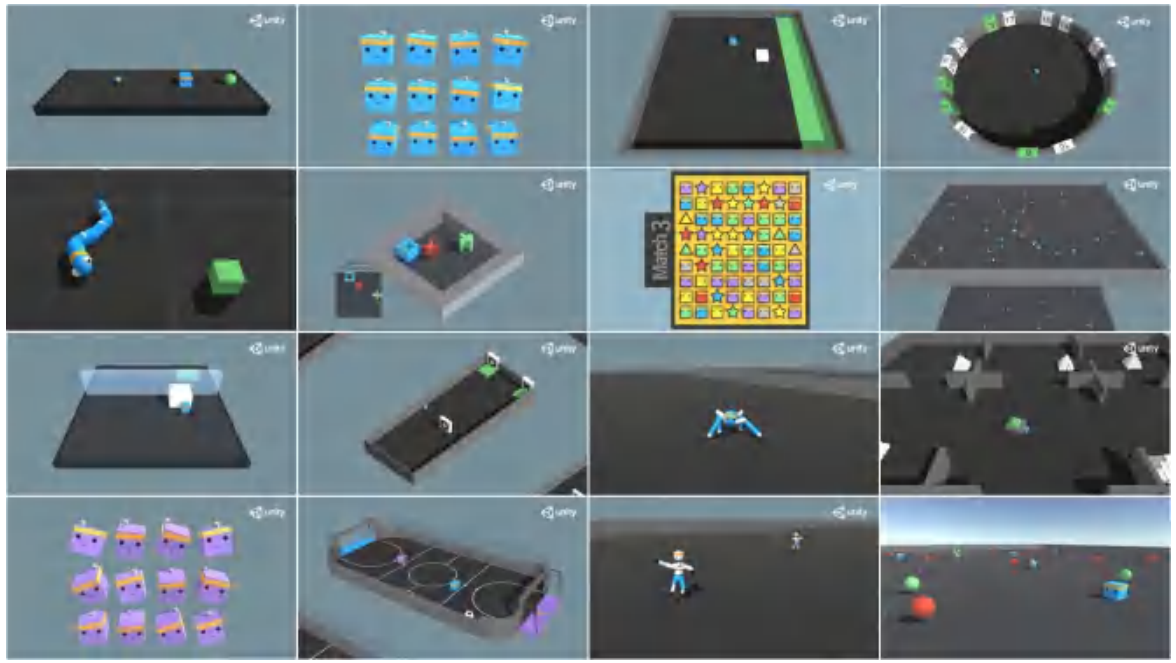
Logic flowchart

Unity ML-Agents is a general platform for agent-based research with powerful environment-building capabilities.
Supports visual complexity, physical simulation complexity, multi-agent interactions, and complex task logic.

https://github.com/Unity-Technologies/ml-agents/blob/main/docs/Learning-Environment-Design-Agents.md https://unity-technologies.github.io/ml-agents/ML-Agents-Overview/

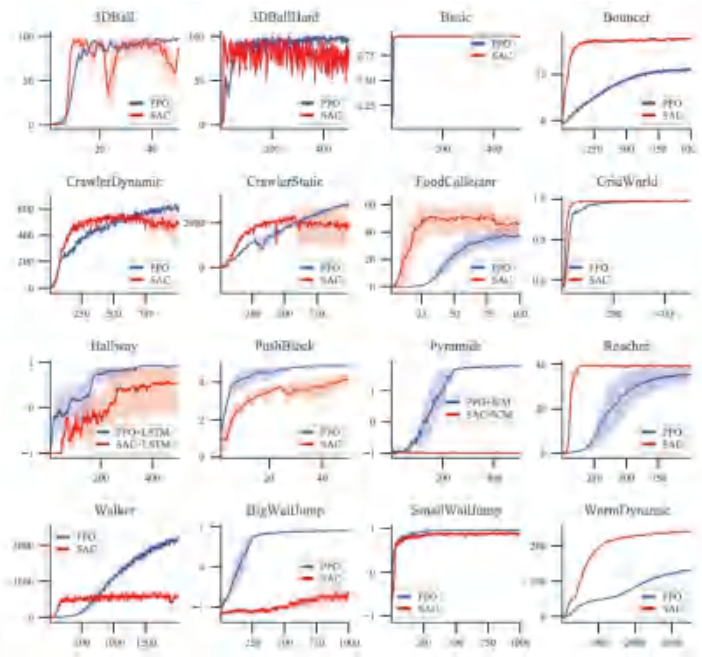
2. Literature Review-MLagent Platform-PPO&SAC

Environment:



Experiment Environments

Training Outcomes and Conclusion:



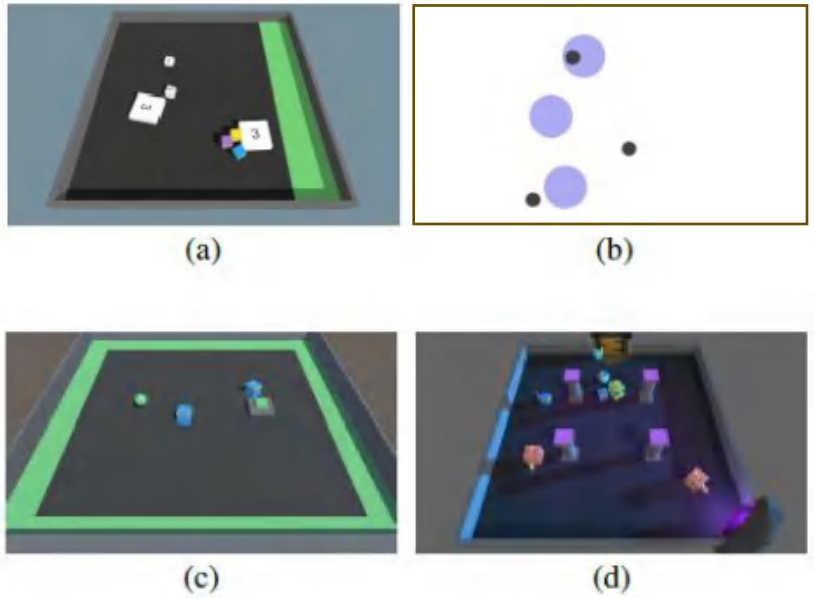
PPO&SAC Training Curve Comparison

Supports multiple **Reinforcement Learning algorithms** (e.g., PPO, SAC, DDPG).
Widely applied in agent **navigation, collaboration, and competition research**.
Unity and ML-Agents platforms meet complex environment requirements for **multi-agent Reinforcement Learning research.**

https://github.com/Unity-Technologies/ml-agents/blob/main/docs/Learning-Environment-Design-Agents.md https://unity-technologies.github.io/ml-agents/ML-Agents-Overview/

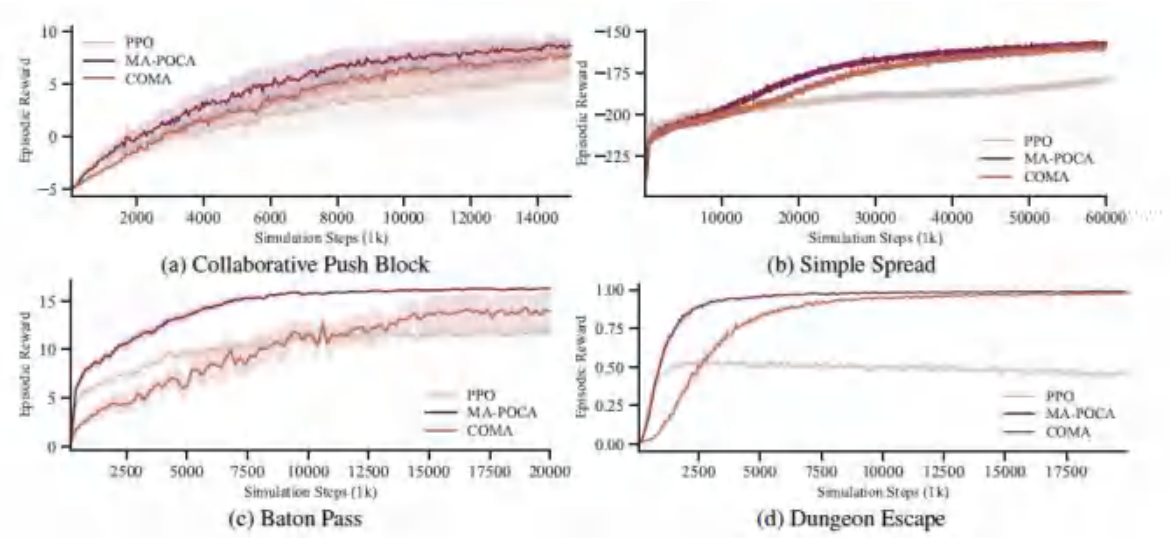
2. Literature Review-MLagent Platform-MAPOCA

Environment:



Experiment Environments

Training Outcomes and Conclusion:



PPO&MAPOCA&COMA Training Curve Comparison

MA-POCA significantly outperforms the traditional MARL structure in tasks with **agent dynamic creation and termination**.
The **attention mechanism**（attention network as Policy network） effectively **reduces the sample complexity** and **improves the overall training efficiency**.
The proposed attention mechanism effectively solves the successor trust allocation and significantly **improves the performance of MARL algorithms** in dynamic environments.

https://github.com/Unity-Technologies/ml-agents/blob/main/docs/Learning-Environment-Design-Agents.md https://unity-technologies.github.io/ml-agents/ML-Agents-Overview/

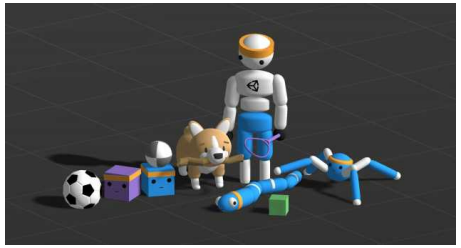
2. Literature Review-Platform Comparison

Overcooked

CREW

Human-AI Teaming

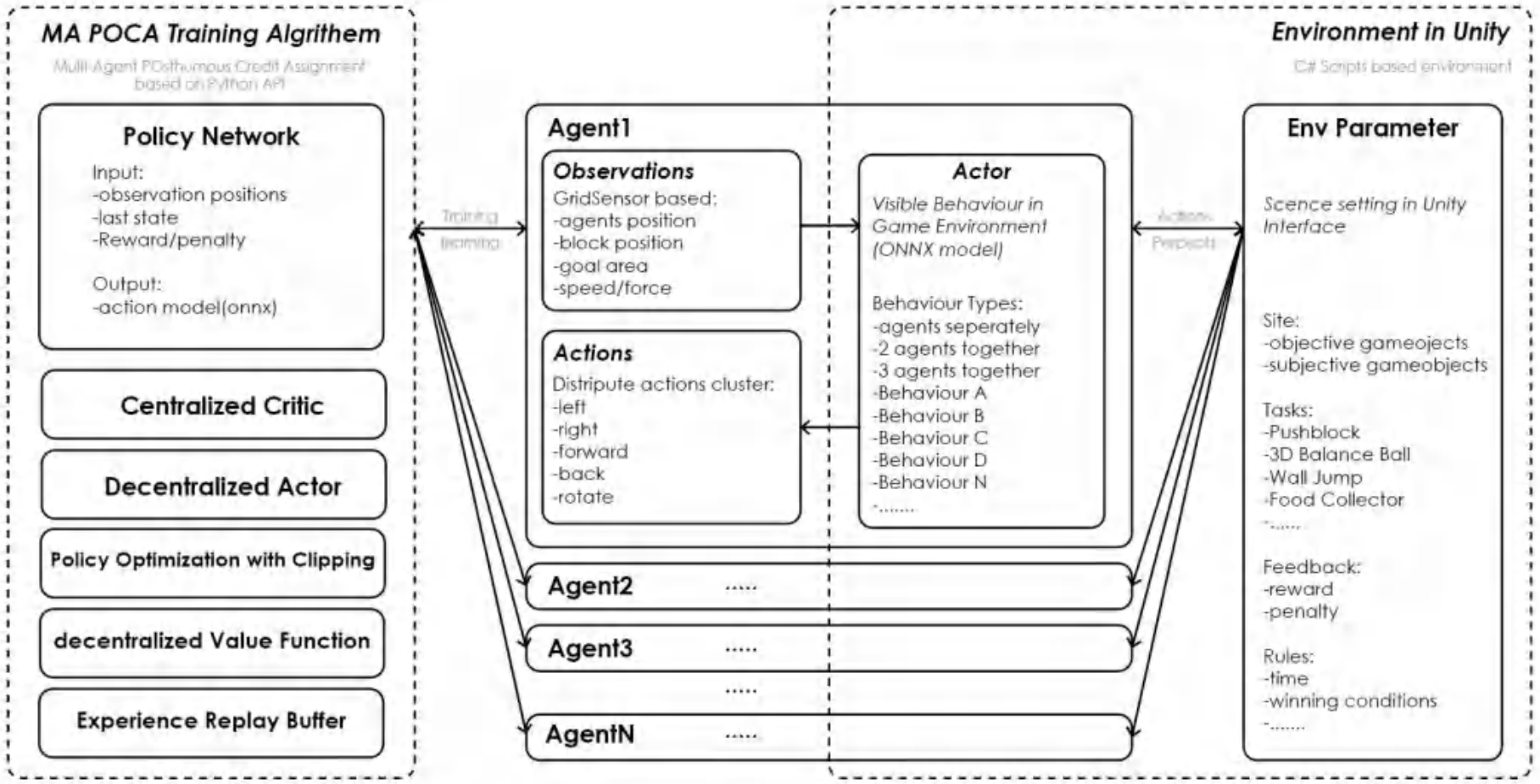
CREW



MLagents

Platform	Extensible envs	Real-time interaction	Multiplayer	Multimodal feedback	Physiological data	Open-sourced	Difficulty
Overcooked	×	√	1+1	√	×	√	Medium
CREW	√	√	No Limit	√	√	√	Difficult
MLagents	√	√	No Limit	√	√	√	Easy

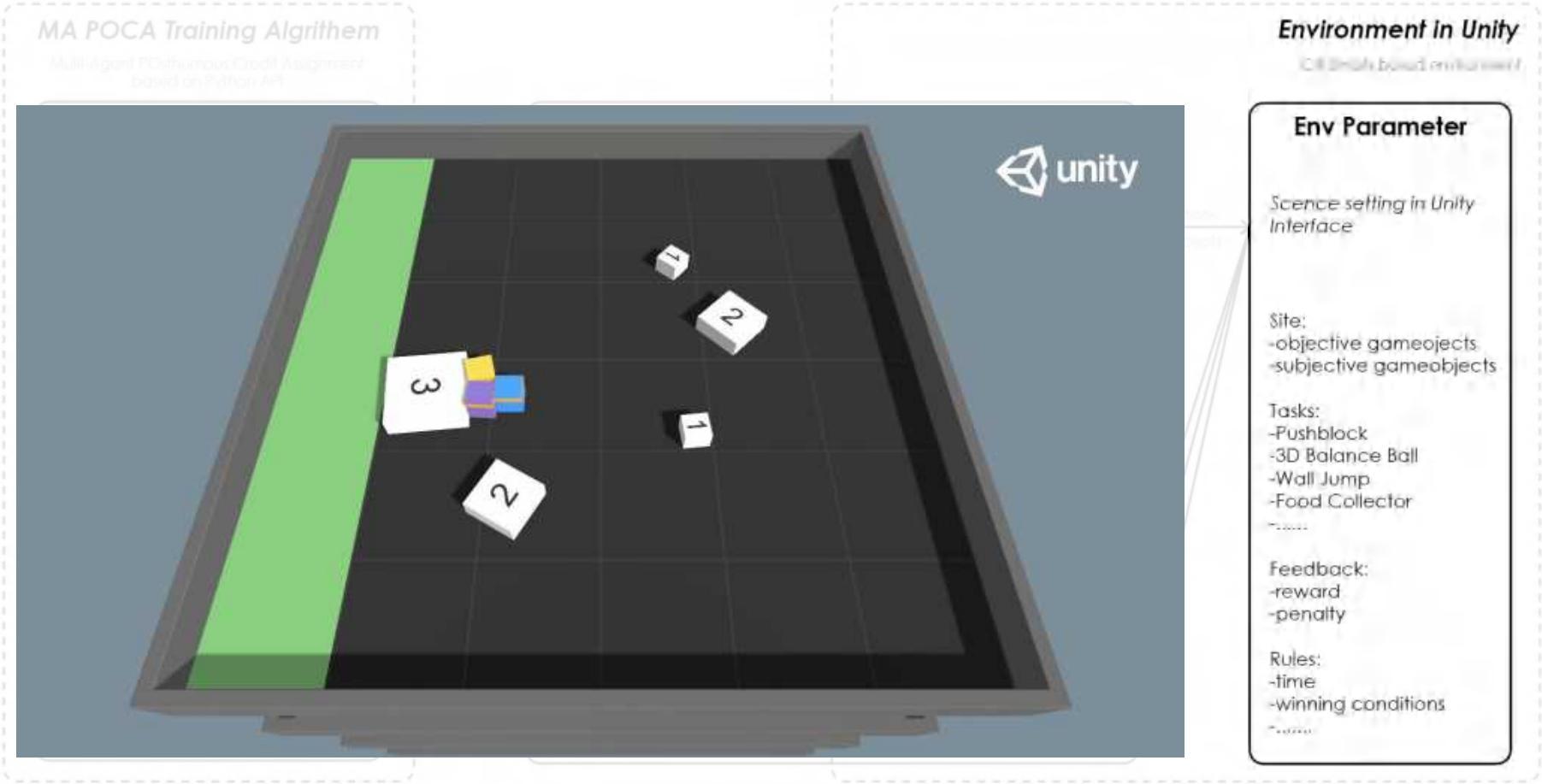
3. Methodology-Workflow



MAPOCA based MARL Workflow

https://en.wikipedia.org/wiki/Multi-agent_system https://en.wikipedia.org/wiki/Multi-agent_reinforcement_learning#Pure_cooperation_settings

3. Methodology-Environment Chosing



Choosing Environment- PushBlockCollab Game

II. Why "Push Block"

1. Game Mechanics

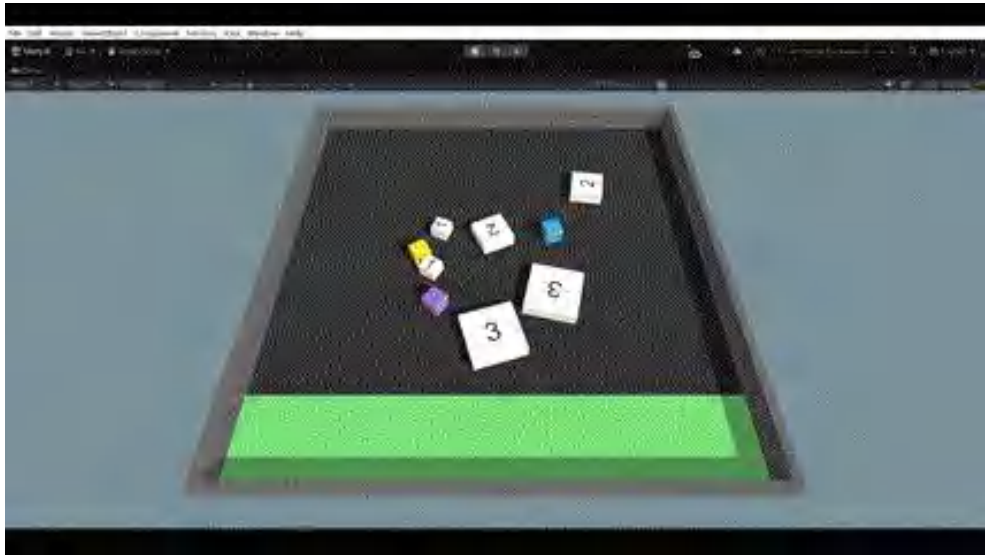
2. Cooperation Challenges

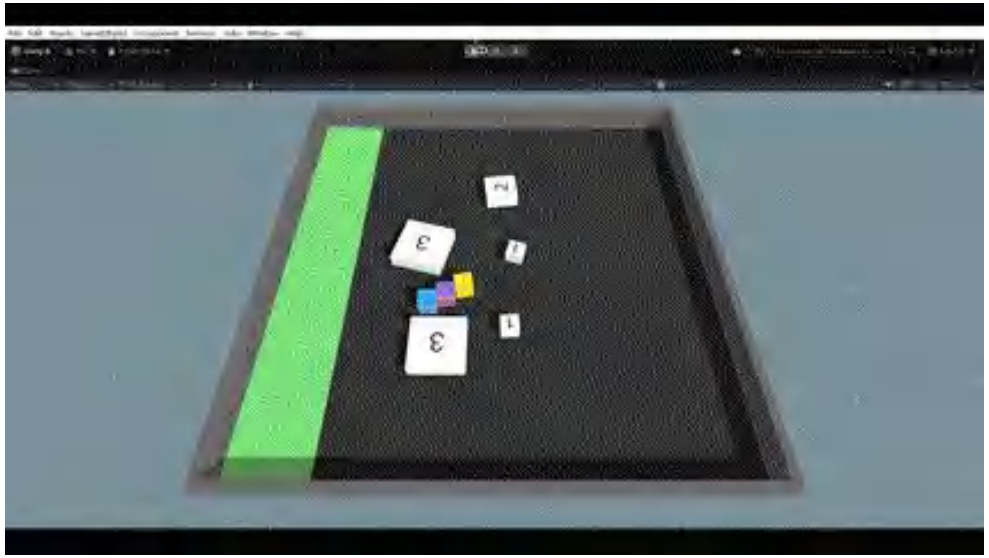
3. Inputs & Outputs

Why The "Push Block"

Push Block: A Platform for Multi-Agent Cooperation

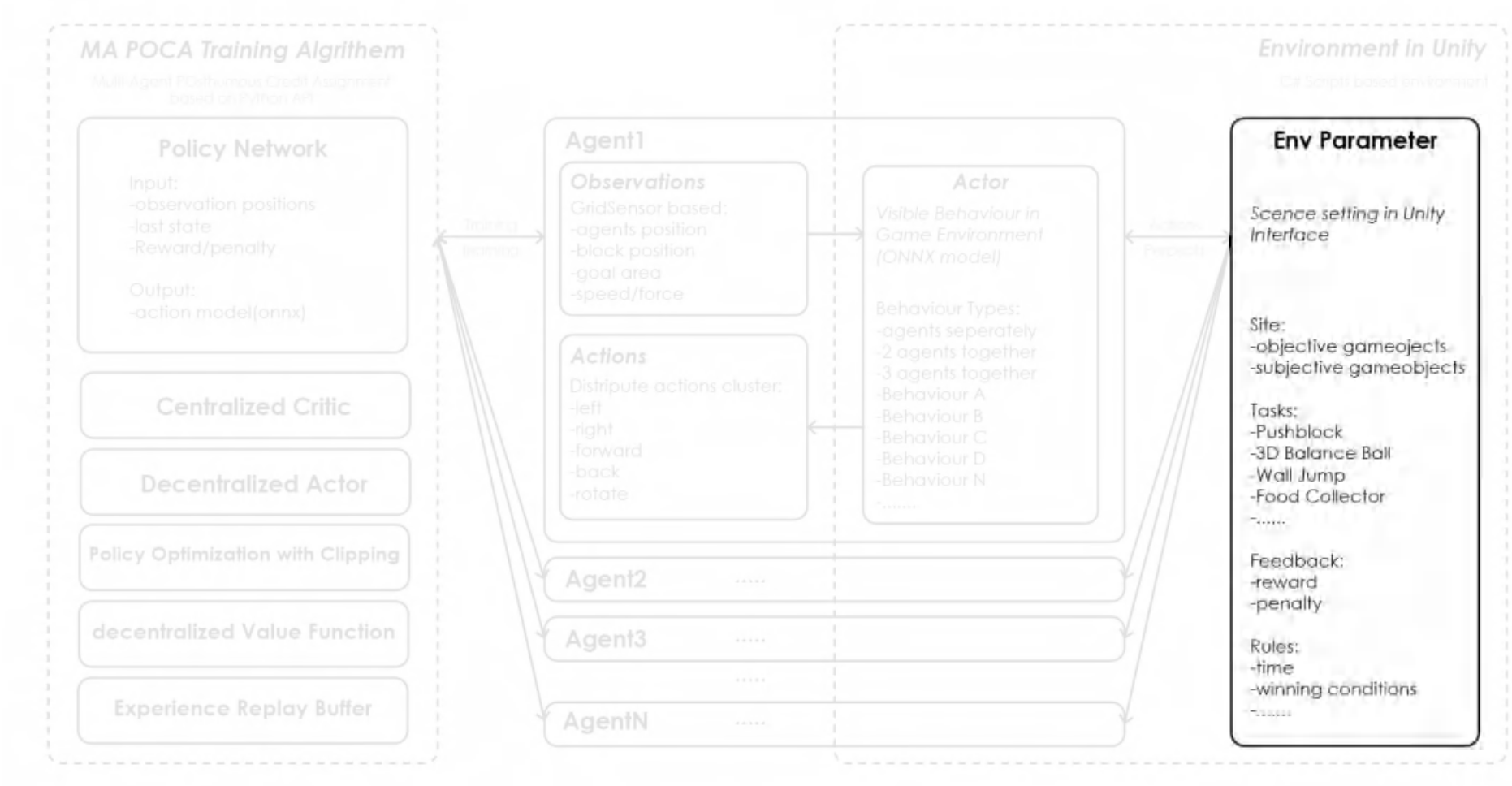
- Clear and simple objective
- Rich cooperation strategies
- Flexible task complexity





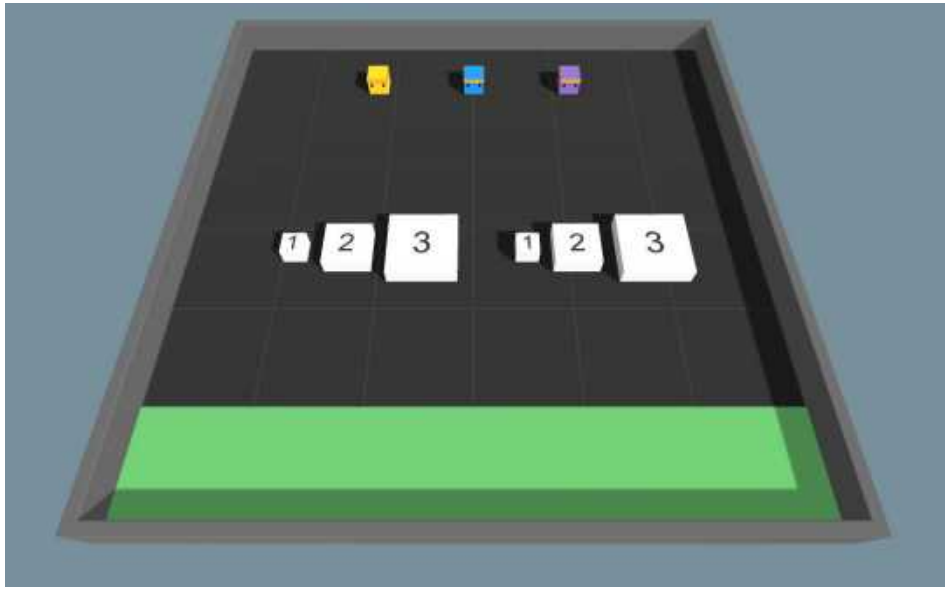
Left image :efficient cooperation, Right image: lack of cooperation

1. Game Mechanics



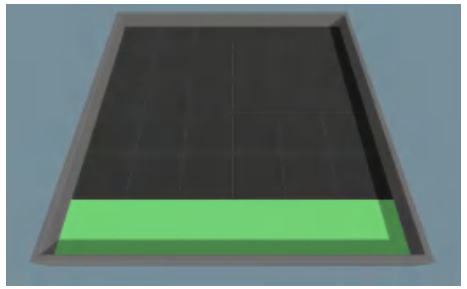
MAPOCA based MARL Workflow

1. Game Mechanics



Episode Conditions

- Agents push all blocks into green Goal Area
- Episode ends with group reward or max steps.



Game Environment

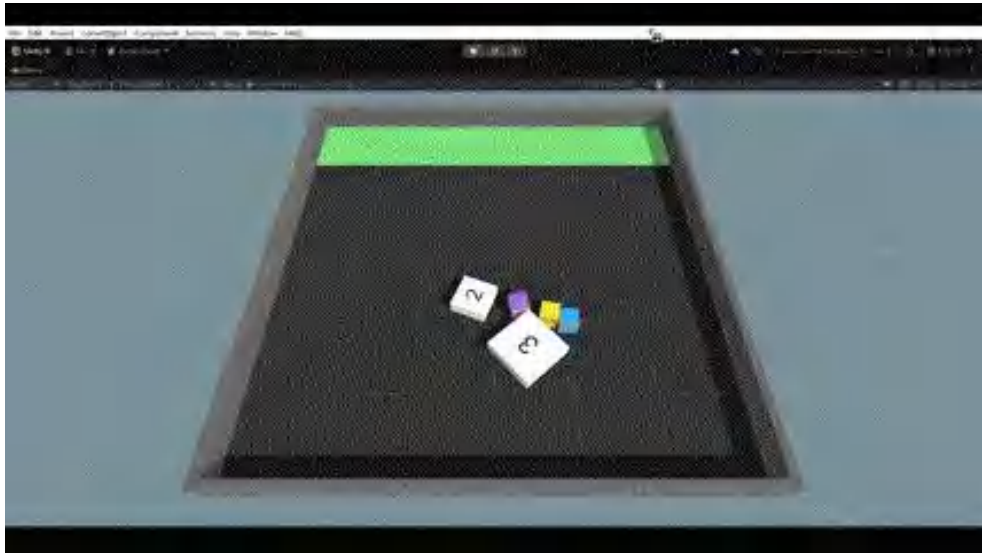
- Ground: Dark grey square
- Goal Area: Fixed green zone
- Walls: Define movement boundaries

	Ground	1	**Small Block**		**Agent**
	Walls	2	**Large Block**		**Agent**
	Goal Area	3	**Very Large Block**		**Agent**

Agents and Blocks

- Agents: 3 colored cubes – Yellow, Blue, Purple
- Blocks: White cubes in 3 sizes, labeled by weight

1. Game Mechanics



	Ground	1	**Small Block**		**Agent**
	Walls	2	**Large Block**		**Agent**
	Goal Area	3	**Very Large Block**		**Agent**

Agents Details

- No communication; independent observation
- Equal strength & max speed
- Movement = facing direction
- Can **push** blocks only (no lifting or passing through)

Block Details

- Small Block (1): Weight = 1; movable by one agent
- Large Block (2): Weight = 2; requires at least two agents
- Very Large Block (3): Weight = 3; requires all agents to cooperate

Goal Area Rules

- Blocks score and disappear when they touch the goal area
- Blocks do not need to fully enter the zone

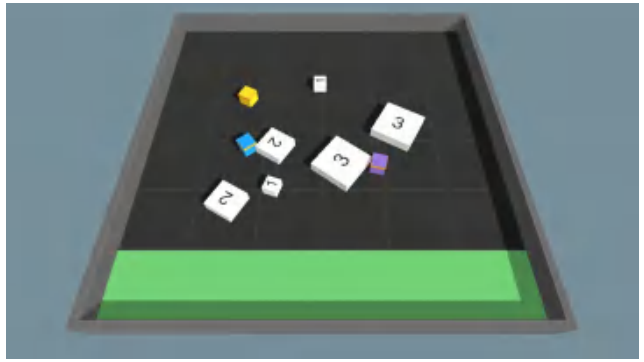
1. Game Mechanics

Initial Setup

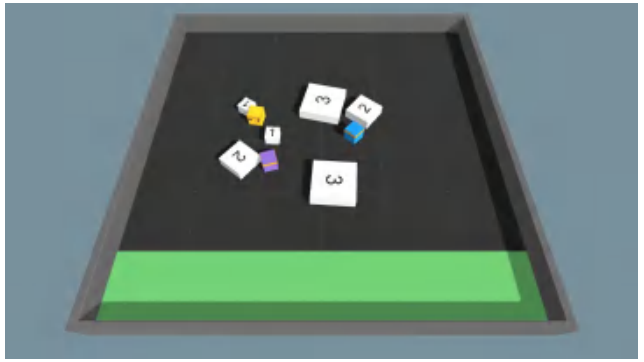
- Random spawn for agents & blocks (non-overlapping, outside goal area)

Reset Conditions

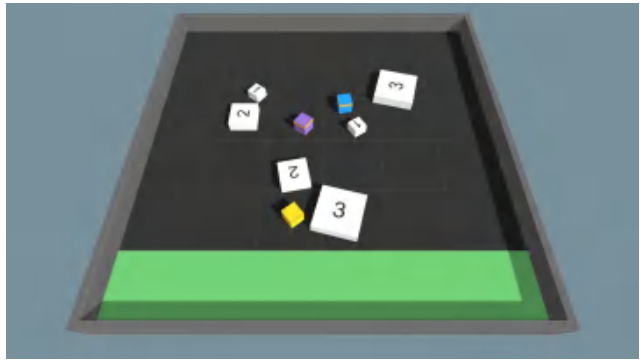
- All blocks reach goal area
- Max steps reached



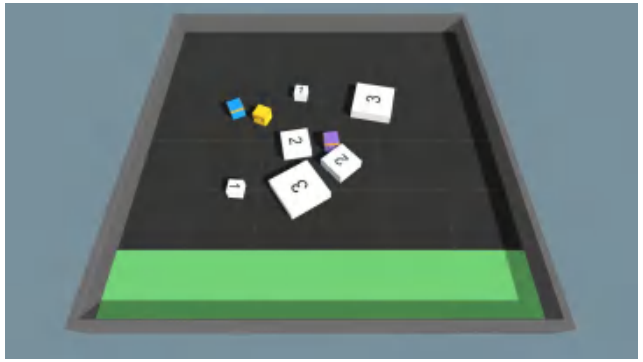
Random Initial Condition A



Random Initial Condition B



Random Initial Condition C



Random Initial Condition D

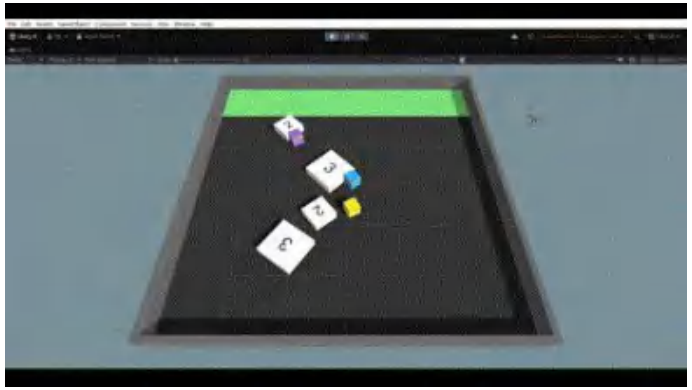
2. Cooperation Challenges

Challenges of Collaboration

- **Incorrect Directions**: Agents act independently, push blocks away from goal
- **Blocks Getting Stuck**: Poor coordination causes blocks to jam
- **Inefficient Cooperation**: Conflicting actions reduce efficiency

Incorrect Directions

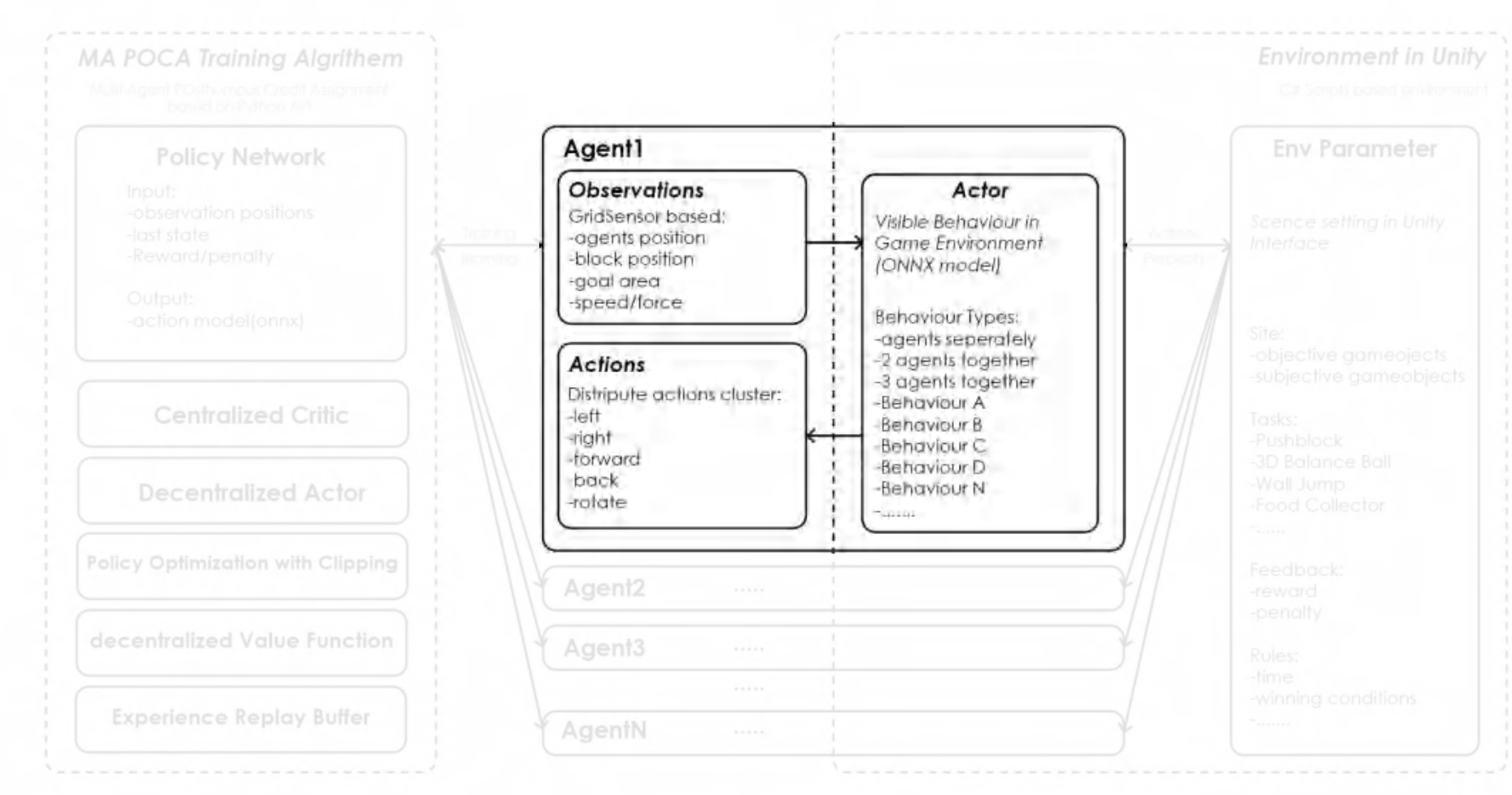
Blocks Getting Stuck



Inefficient Cooperation

Tasks require teamwork, not individual effort.

3. Inputs & Outputs



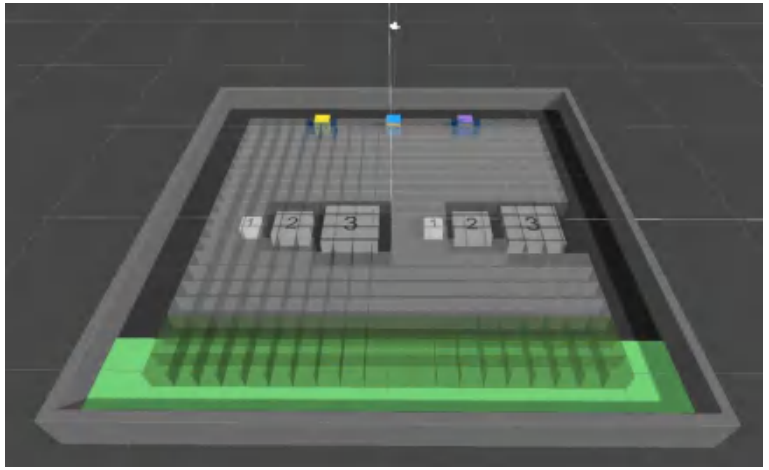
MAPOCA based MARL Workflow

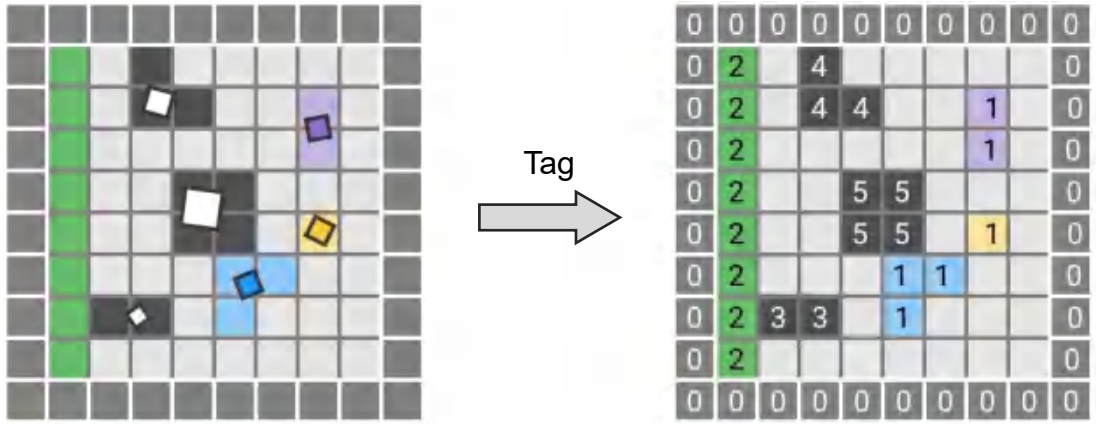
3. Inputs & Outputs

Grid Inputs (CNN-based Visual Perception)

- Agent observes surroundings using 2D grid
- Each grid cell encoded with one-hot vector
- Forms a 3D tensor → fed into CNN policy
- Observation size:

$$GridSize.x \times GridSize.z \times NumDetectableTags$$

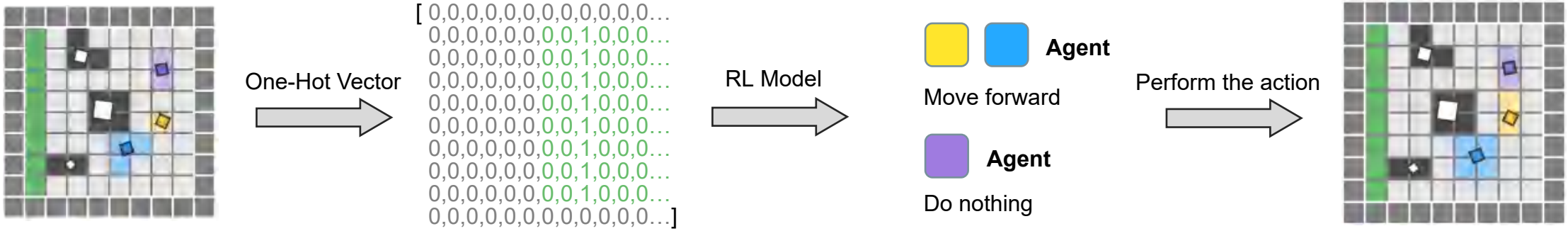




Reference visual: ML-Agents visual observation structure

3. Inputs & Outputs

Observe Environment→ Select Action→ Perform Action→ Receive Feedback (Next State)



Detectable Tags: 6

Tag	One-Hot Vector	Object Category
• Tag N	[0, 0, 0, 0, 0, 0]	Nothing
• Tag 0	[1, 0, 0, 0, 0, 0]	Wall
• Tag 1	[0, 1, 0, 0, 0, 0]	Agent
• Tag 2	[0, 0, 1, 0, 0, 0]	Goal
• Tag 3	[0, 0, 0, 1, 0, 0]	BlockSmall
• Tag 4	[0, 0, 0, 0, 1, 0]	BlockLarge
• Tag 5	[0, 0, 0, 0, 0, 1]	BlockVeryLarge

Action Outputs

- 0: Do nothing
- 1: Move forward
- 2: Move backward
- 3: Rotate clockwise
- 4: Rotate counterclockwise
- 5: Move left
- 6: Move right

III. ML Agents RL

1. Collaboration Hypothesis

2. Customized Reward Function

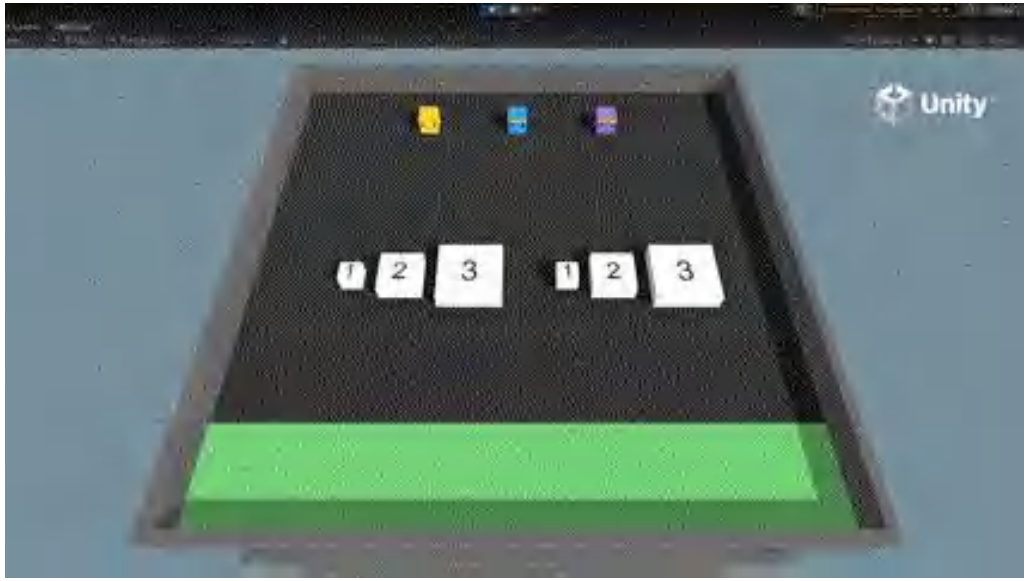
2. Customized Reward Function

Original PushBlcokCollab logic

```
public void ScoredAGoal(Collider col, float score)
{
    print($"Scored {score} on {gameObject.name}");

    // Decrease the number of remaining blocks
    m_NumberOfRemainingBlocks--;

    // Check if all tasks are completed
    bool done = m_NumberOfRemainingBlocks == 0;

    // Disable the block
    col.gameObject.SetActive(false);

    // Give a fixed reward
    m_AgentGroup.AddGroupReward(score);

    // Change ground material as visual feedback
    StartCoroutine(GoalScoredSwapGroundMaterial(
        m_PushBlockSettings.goalScoredMaterial, 0.5f));

    // If all blocks have been pushed into the goal, end the episode
    if (done)
    {
        m_AgentGroup.EndGroupEpisode();
        ResetScene();
    }
}
```

Decrease Number Blocks

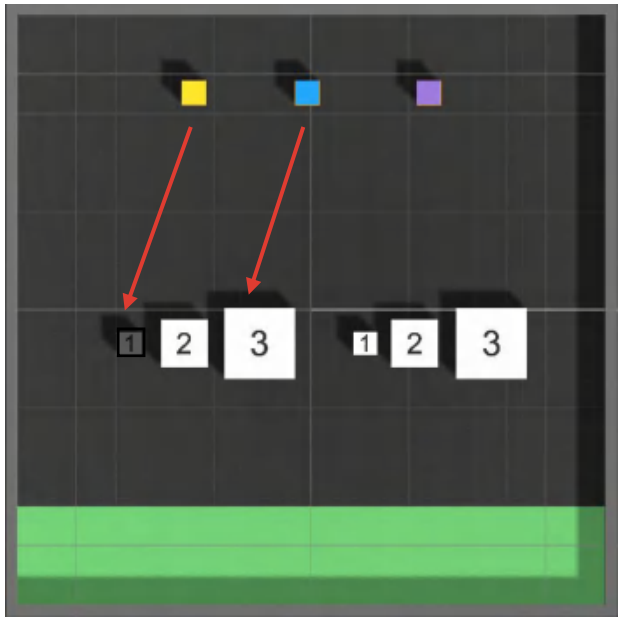
All blocks are pushed in the goal zone, end the episode



1. Collaboration Hypotheses

Original PushBlcokCollab logic

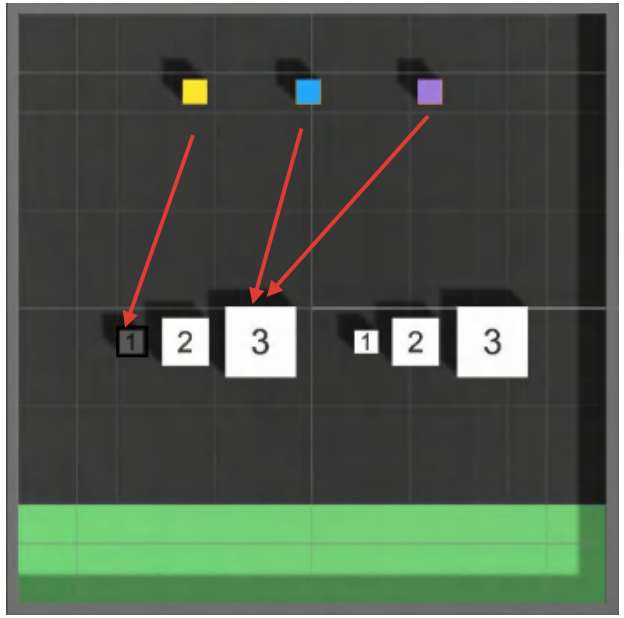
Scenario 1 -- Individual



Energy Bar

Yellow Agent		-1
Blue Agent		-1
Purple Agent		0
Block Count		-1
Reward		+1

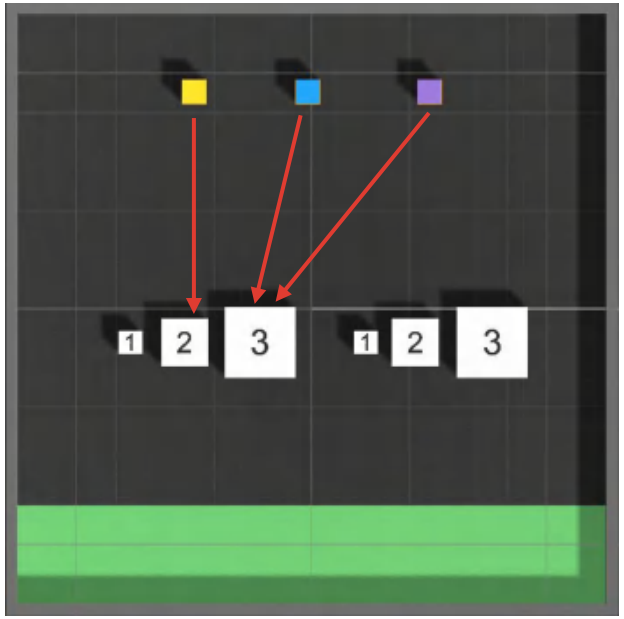
Scenario 2 -- Individual



Energy Bar

Yellow Agent		-1
Blue Agent		-1
Purple Agent		-1
Block Count		-1
Reward		+1

Scenario 3 -- Individual



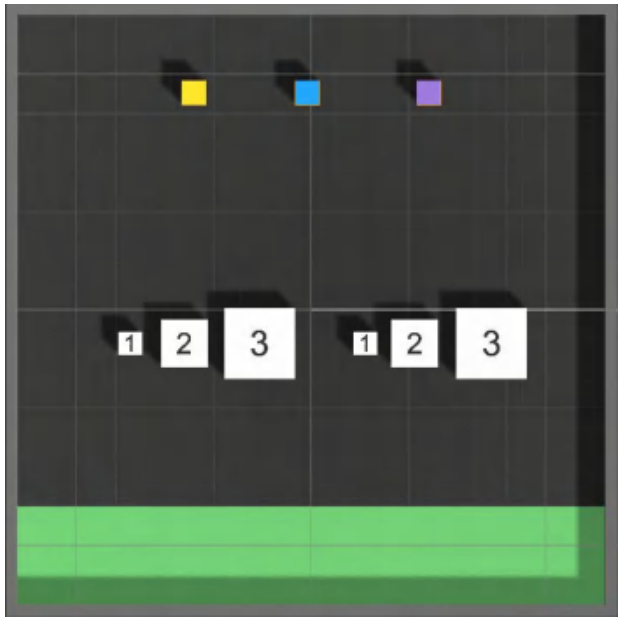
Energy Bar

Yellow Agent		-1
Blue Agent		-1
Purple Agent		-1
Block Count		0
Reward		0

1. Collaboration Hypotheses

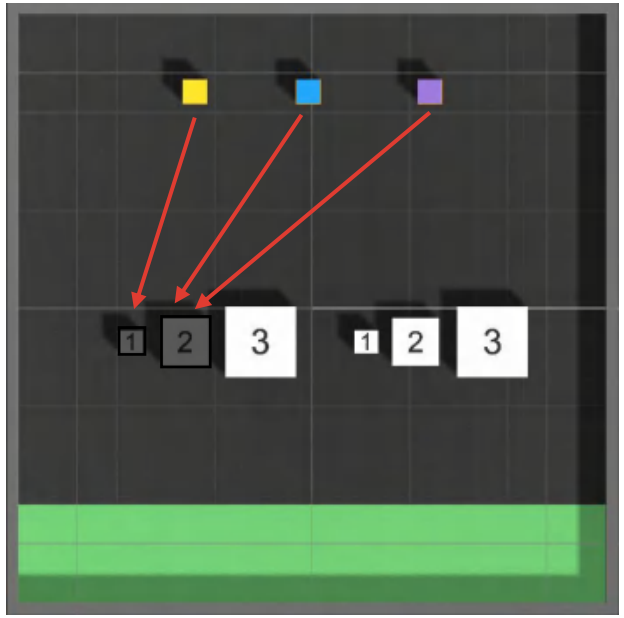
Basis of reward function logic

Initial Scene

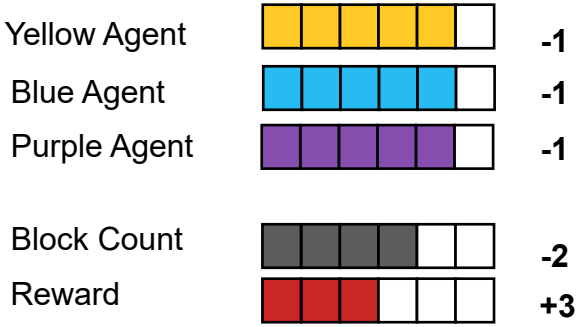


Energy Bar

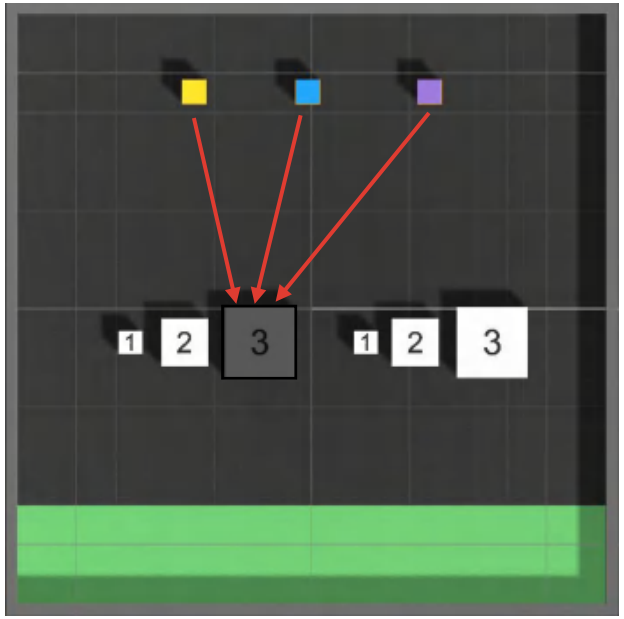
Hypothesis 1 -- Collaboration



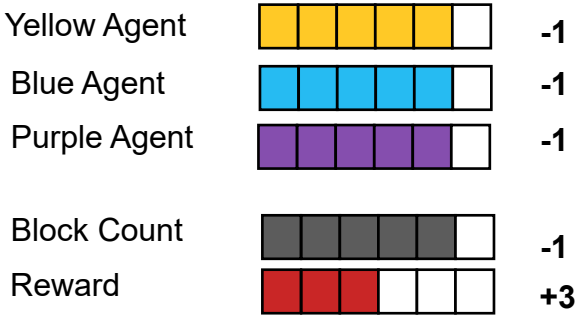
Energy Bar



Hypothesis 2 -- Collaboration



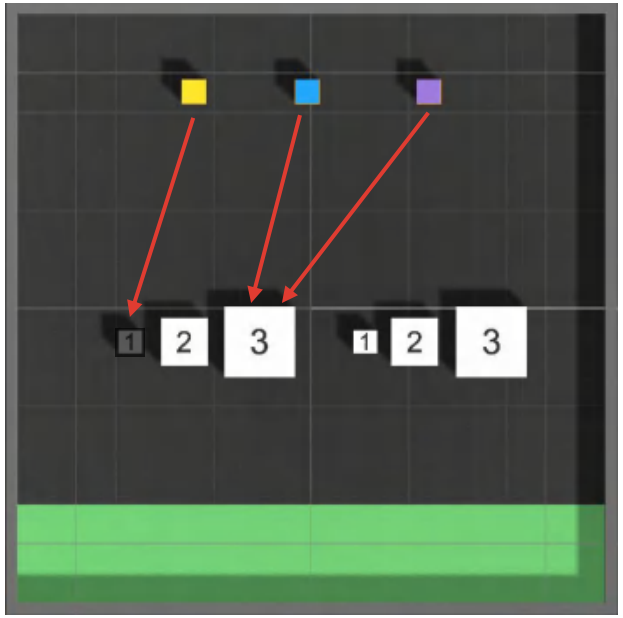
Energy Bar



1. Collaboration Hypotheses

Basis of reward function logic

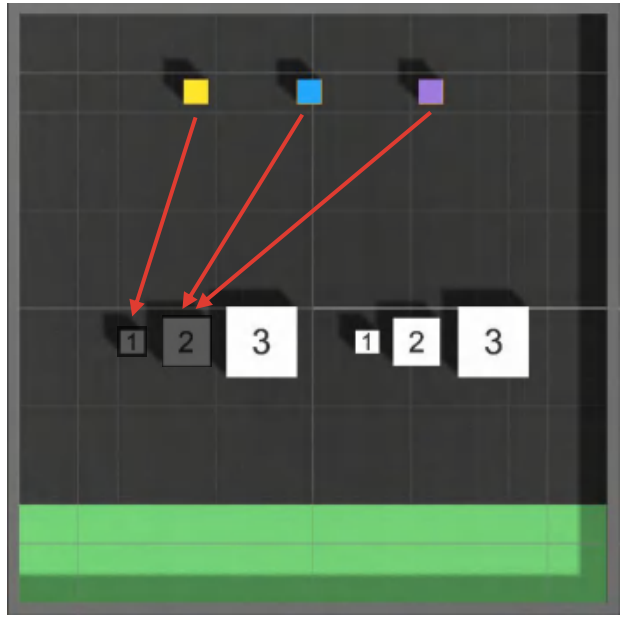
Default Reward Function -- Individual



Energy Bar

Yellow Agent		-1
Blue Agent		-1
Purple Agent		-1
Block Count		-1
Reward		+1

Customized Reward Function-- Collaboration



Energy Bar

Yellow Agent		-1
Blue Agent		-1
Purple Agent		-1
Block Count		-2
Reward		+3

2. Customized Reward Function

Logic Construction – Agent Matching

Rewards for blocks pushed into score zone: provides different rewards based on the **block type**

in *OnBlockGoalScored()* Method

Maximum rewards for completing collaborative tasks:

- **Block3: 3** agents to collaborate
  Awards **A** points
- **Block2: 2** agents to collaborate
  Awards **B** points
- **Block1: 1** agents to push
  Awards **C** points

```
// Set the number of required agents and rewards according to the different Block types.
int requiredAgents = 0;
float rewardValue = 0f;
string blockTypeName = "";

switch (blockType)
{
    case BlockTypeIdentifier.BlockType.Block3:
        requiredAgents = 3;
        rewardValue = 15f;
        blockTypeName = "Block3";
        break;
    case BlockTypeIdentifier.BlockType.Block2:
        requiredAgents = 2;
        rewardValue = 10f;
        blockTypeName = "Block2";
        break;
    default: // Block1
        requiredAgents = 1;
        rewardValue = 7f;
        blockTypeName = "Block1";
        break;
}
```

ABC Experimental Data

2. Customized Reward Function

Construction Logic – Agent Matching

Agent Count Validation:

Checks whether the number of agents participating in pushing the block meets the requirements.

$$R_{\text{collab}} = \begin{cases} -2.0, & A_{\text{active}} = 0 \\ R_{\max}, & A_{\text{active}} = A_{\text{required}} \\ -1.0 \times (A_{\text{required}} - A_{\text{active}}), & A_{\text{active}} < A_{\text{required}} \end{cases}$$

A_{active} → Number of agents actually involved in pushing the block

A_{required} → Minimum number of agents required to push the block

$R_{\max}$ → Maximum rewards for completing collaborative tasks

```
// Get the number of active agents collaborating
int activeAgents = contributionTracker.GetActiveAgentCount();

// Record all contributing agents' details
Dictionary<int, float> allContributions = contributionTracker.GetAllContributions();

float totalAgentReward = 0f;
bool validAgentCount = (activeAgents == requiredAgents);

if (activeAgents == 0)
{
    // Apply a negative reward if no agents contributed
    totalAgentReward = -2f;
}
else if (validAgentCount)
{
    // Correct number of agents collaborated, grant full reward
    totalAgentReward = rewardValue;
}
else
{
    // Mismatched agent count, apply partial penalty
    totalAgentReward = -1f * (requiredAgents - activeAgents);
}

// Apply the reward
m_AgentGroup.AddGroupReward(totalAgentReward);

// Reset contribution tracker
contributionTracker.ResetContributions();
```

2. Customized Reward Function

Construction Logic – Time Penalty

Negative Rewards for Time Passing: In each *FixedUpdate()*, all agents receive a small negative reward to encourage them to complete the task quickly:

$$R_{\text{time}} = -\frac{0.05}{\text{MaxEnvironmentSteps}}$$

```
void FixedUpdate()
{
    m_ResetTimer++;
    // If it hasn't finished by the timeout, force it to end
    if (m_ResetTimer >= MaxEnvironmentSteps && MaxEnvironmentSteps > 0)
    {
        m_AgentGroup.GroupEpisodeInterrupted();
        ResetScene();
    }

    // Negative bonus for passing time to encourage agents to finish quickly
    m_AgentGroup.AddGroupReward(-0.05f / MaxEnvironmentSteps);
}
```

2. Customized Reward Function

Construction Logic – Total Cooperation Measurement

Collaboration Reward $R_{collab:}$

$$R_{\text{collab}} = \begin{cases} -2.0, & A_{\text{active}} = 0 \\ R_{\max}, & A_{\text{active}} = A_{\text{required}} \\ -1.0 \times (A_{\text{required}} - A_{\text{active}}), & A_{\text{active}} < A_{\text{required}} \end{cases}$$

Time Penalty $R_{time:}$

$$R_{\text{time}} = -\frac{0.05}{\text{MaxEnvironmentSteps}}$$

$$R_{\text{total}} = R_{\text{collab}} + R_{\text{time}}$$

A_{active} → Number of agents actually involved in pushing the block

$A_{required}$ → Minimum number of agents required to push the block

R_{max} → Maximum rewards for completing collaborative tasks

R_{time} → Penalty to encourage agents to finish tasks fastly

Ⅳ. Result Analysis

1. Analyze Results — by Reward & Entropy
2. Optimize Results
3. Conclusion
4. Future Works

1. Analyze the Result — Reward / Entropy

Larger Entropy == Bigger Randomness

Larger Group Rewards == Better Collaboration (reward function)

Two special situations

Block 3 awards 5
Block 2 awards 3
Block 1 awards 2

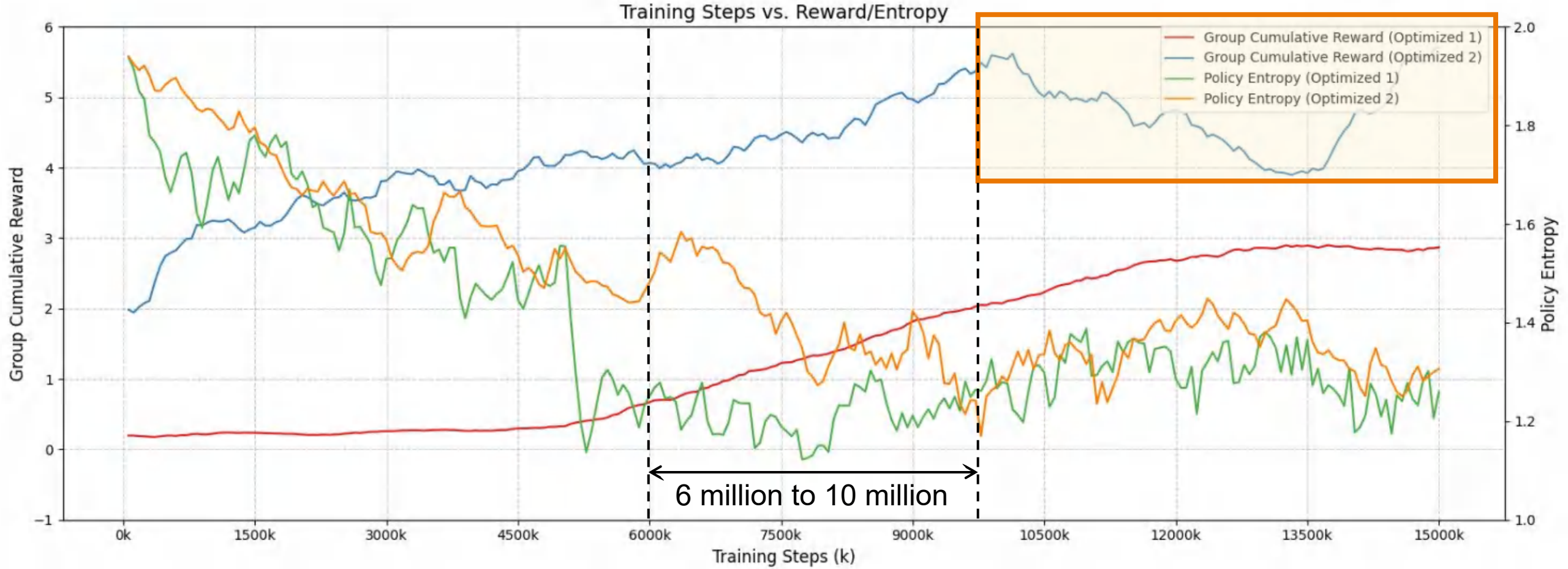
2. Optimize the Result — Reward / Entropy

Original : Mass for Block 1-100, Block 2 – 300, Block 3 – 600

Optimized 1: Mass for Block 1-10, Block 2 – 200, Block 3 – 300

Optimized 2: Mass for Block 1-10, Block 2 – 100, Block 3 – 150

Block 3 awards 5
Block 2 awards 3
Block 1 awards 2



2. Optimize the Result — Flexible Reward Function

More Proactive Agent Matching

Piecewise processing creates ***sharp thresholds*** at key points, unable to gracefully handle near-miss scenarios.

Rewards now ***scale linearly*** with how far agent numbers are from the goal ($|A_{required} - A_{active}|$), avoiding hard switches.

$$R_{\text{collab}} = \begin{cases} -2.0, & A_{\text{active}} = 0 \\ R_{\max}, & A_{\text{active}} = A_{\text{required}} \\ -1.0 \times (A_{\text{required}} - A_{\text{active}}), & A_{\text{active}} < A_{\text{required}} \end{cases}$$

$$R_{collab} = \frac{R_{max} + \left(1 - 0.5 \times |A_{required} - A_{active}|\right)}{2}$$

A_{active} → Number of agents actually involved in pushing the block

$A_{required}$ → Minimum number of agents required to push the block

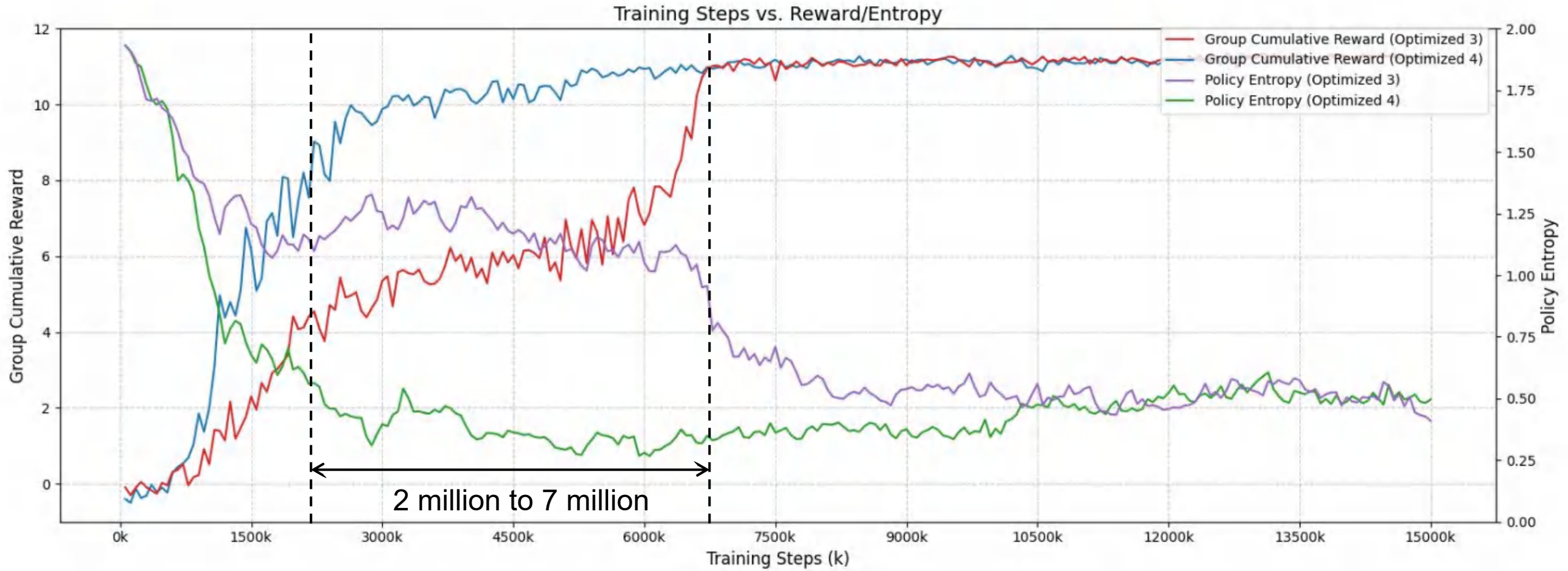
R_{max} → Maximum rewards for completing collaborative tasks

2. Optimize the Result — Reward / Entropy

Optimized 3: Mass for Block 1-10, Block 2 – 40, Block 3 – 90

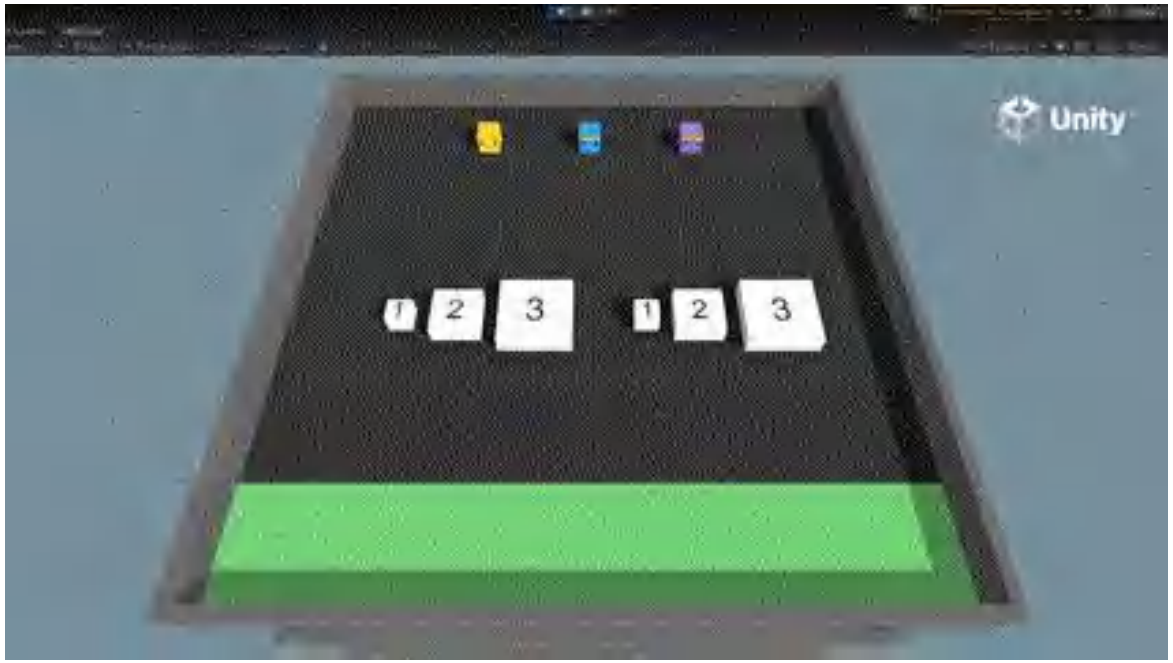
Optimized 4: Mass for Block 1-10, Block 2 – 60, Block 3 – 100

Block 3 awards 5
Block 2 awards 3
Block 1 awards 1

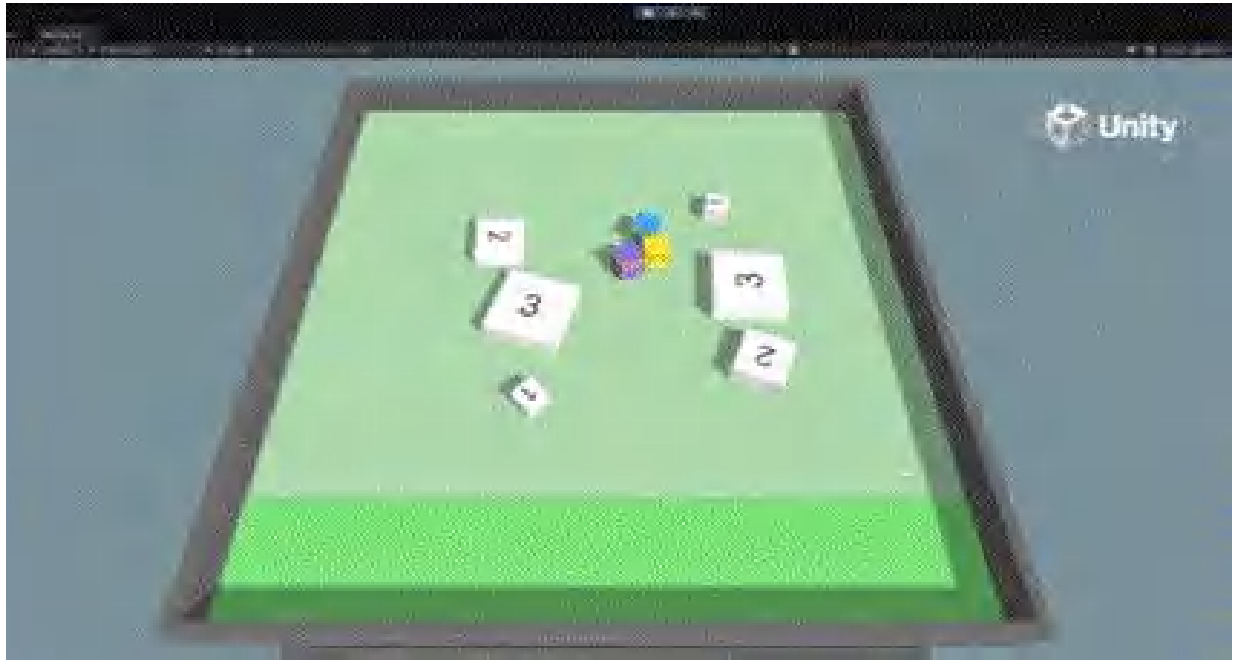


3. Conclusion — Training Outcome

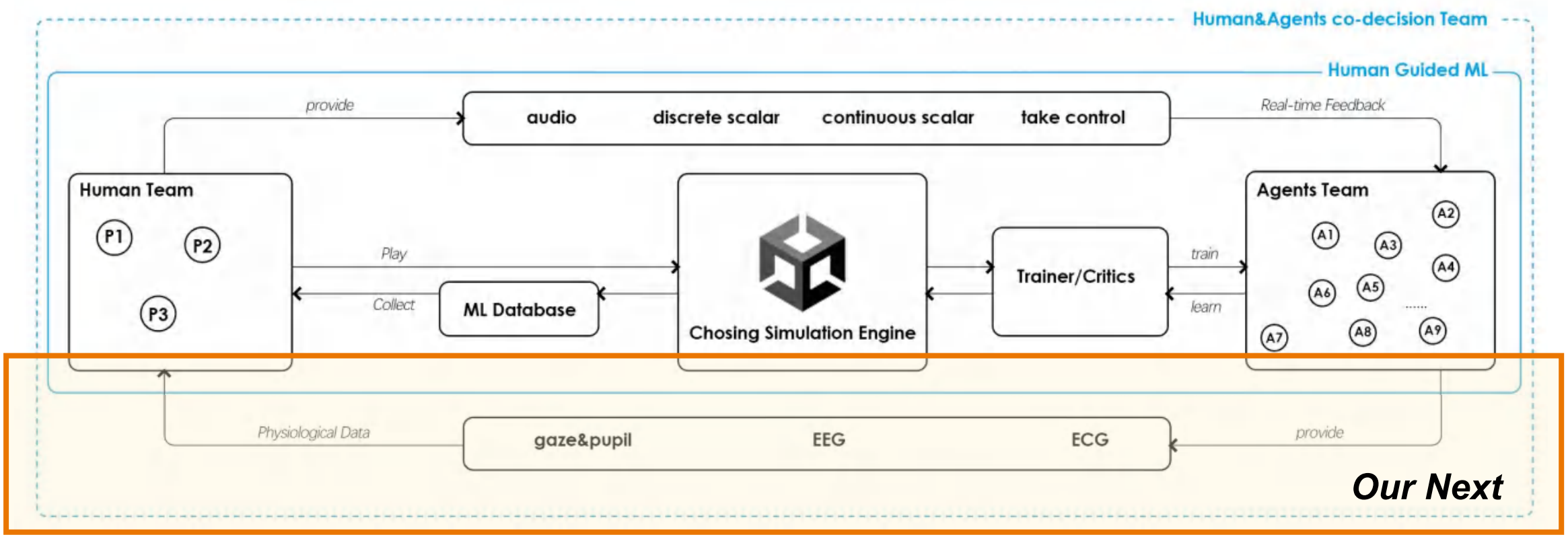
Default Training Outcome



Customized Training Outcome



4. Future Works



Abstracted Human-Agent Collaboration **Simulation Prototype**

Reference

• Juliani, Arthur, Vincent-Pierre Berges, Ervin Teng, Andrew Cohen, Jonathan Harper, Chris Elion, Chris Goy, et al. “Unity: A General Platform for Intelligent Agents.” *arXiv* preprint arXiv:1809.02627v2, May 6, 2020. https://arxiv.org/abs/1809.02627.

• Cohen, Andrew, Ervin Teng, Vincent-Pierre Berges, Ruo-Ping Dong, Hunter Henry, Marwan Mattar, Alexander Zook, and Sujoy Ganguly. “On the Use and Misuse of Absorbing States in Multi-agent Reinforcement Learning.” *arXiv* preprint arXiv:2111.05992v2, June 7, 2022. https://arxiv.org/abs/2111.05992.

• Le Pelletier de Woillemont, Pierre, Rémi Labory, and Vincent Corruble. “Automated Play-Testing Through RL Based Human-Like Play-Styles Generation.” *arXiv* preprint arXiv:2211.17188v1, November 29, 2022. https://arxiv.org/abs/2211.17188.

• Zhang, Lingyu, Zhengran Ji, and Boyuan Chen. “CREW: Facilitating Human-AI Teaming Research.” *arXiv* preprint arXiv:2408.00170v1, July 31, 2024. http://generalroboticslab.com/CREW.

• Carroll, Micah, Rohin Shah, Mark K. Ho, Thomas L. Griffiths, Sanjit A. Seshia, Pieter Abbeel, and Anca Dragan. “On the Utility of Learning about Humans for Human-AI Coordination.” *arXiv* preprint arXiv:1910.05789v2, January 9, 2020. https://arxiv.org/abs/1910.05789.